# tie 语言语法规范

tie Language Specification

版本 2026.2

---

本文档的正文内容与渲染实现均为 tie 语言源码，由 tofflib 生成。

# 目录


第 1 章 词法与记号 ..... 6
1.1 源文件结构：头部与内容 ..... 6
1.2 文件角色 ..... 6
1.3 字符集与编码 ..... 7
1.4 注释 ..... 8
1.5 标识符 ..... 8
1.6 关键字 ..... 8
1.7 数值字面量 ..... 8
1.8 布尔与三值字面量 ..... 9
1.9 字符与字符串字面量 ..... 9
1.10 自动分号 ..... 10
1.11 行续行 ..... 10
1.12 运算符记号 ..... 11
1.13 上下文关键字 ..... 11

第 2 章 类型系统 ..... 12
2.1 基本类型 ..... 12
2.2 平衡三进制 trit ..... 13
2.3 宽类型 ..... 13
2.4 表类型 ..... 13
2.5 映射类型 ..... 14
2.6 元组类型 ..... 14
2.7 类型标注与推导 ..... 14
2.8 泛型 ..... 15
2.9 枚举 ..... 16
2.10 动态类型 ..... 16
2.11 类型转换 ..... 16
2.12 低层与特殊类型 ..... 17
2.13 定长数组 ..... 17
2.14 函数类型 ..... 17
2.15 行池 ..... 18
2.16 精确数值类型 ..... 18
2.17 类型别名 ..... 18
2.18 泛型的值参数 ..... 18
2.19 字符串处理 ..... 19
2.20 集合操作 ..... 19

第 3 章 运算符与表达式 ..... 20
3.1 优先级与结合性 ..... 20
3.2 算术运算 ..... 21
3.3 整除与成对取余 ..... 21
3.4 幂运算 ..... 22
3.5 带检查的运算 ..... 22
3.6 比较运算 ..... 22
3.7 逻辑运算与三值逻辑 ..... 23
3.8 位运算 ..... 23
3.9 成员判定 ..... 23
3.10 加法的语义扩展 ..... 23
3.11 数据流箭头 ..... 24
3.12 不可变更新 ..... 26
3.13 可空链 ..... 26
3.14 解引用 ..... 27
3.15 范围与切片 ..... 27
3.16 类型判定 ..... 27
3.17 集合综合 ..... 28

第 4 章 语句 ..... 28

4.1 变量声明 ..... 28
4.2 赋值 ..... 28
4.3 条件分支 ..... 29
4.4 循环 ..... 29
4.5 多分支选择 ..... 30
4.6 跳转 ..... 30
4.7 表达式语句 ..... 31
4.8 条件解构 ..... 31
4.9 前置条件 ..... 31
4.10 作用域收尾 ..... 31
4.11 错误聚合块 ..... 32
4.12 跳转与标签 ..... 32
4.13 分支即表达式 ..... 32
4.14 解构 ..... 33

第 5 章 函数 ..... 33
5.1 定义与调用 ..... 33
5.2 参数传递约定 ..... 33
5.3 默认参数 ..... 34
5.4 命名实参 ..... 35
5.5 变长参数 ..... 35
5.6 返回值 ..... 35
5.7 一等函数 ..... 36
5.8 闭包 ..... 36
5.9 短闭包 ..... 36
5.10 捕获面 ..... 36
5.11 外部函数 ..... 37
5.12 等号体与表达式体 ..... 37
5.13 隐式单参闭包 ..... 37
5.14 实参展开 ..... 38

第 6 章 数据结构 ..... 38
6.1 结构的定义 ..... 38
6.2 构造与字段访问 ..... 38
6.3 逻辑与数据分离 ..... 38
6.4 方法调用 ..... 39
6.5 继承 ..... 39
6.6 语义限制 ..... 40
6.7 与元组的关系 ..... 40
6.8 运算约定 ..... 40
6.9 计算属性 ..... 41
6.10 遍历约定 ..... 41
6.11 迭代器适配 ..... 42

第 7 章 接口 ..... 42
7.1 声明 ..... 42
7.2 实现 ..... 43
7.3 作为约束 ..... 43
7.4 作为对象 ..... 43
7.5 接口的组合 ..... 44
7.6 两种接口 ..... 44

第 8 章 错误处理 ..... 44
8.1 可预期的失败 ..... 45
8.2 传播 ..... 45
8.3 分支处理 ..... 45
8.4 标准组合子 ..... 46
8.5 不可恢复的故障 ..... 46
8.6 错误信息 ..... 46
8.7 错误链 ..... 46

第 9 章 模块与导入 ..... 47
9.1 导入 ..... 47
9.2 命名空间 ..... 47
9.3 可见性 ..... 47
9.4 标准库 ..... 48
9.5 依赖与版本 ..... 49
9.6 选择性导入与重导出 ..... 49
9.7 引入命名空间 ..... 49
9.8 数据格式 ..... 49
9.9 依赖声明 ..... 50

第 10 章 并发 ..... 50
10.1 并发的基本立场 ..... 50
10.2 状态单元 ..... 50
10.3 消息与返回 ..... 51
10.4 任务与依赖 ..... 51
10.5 与闭包捕获的关系 ..... 51
10.6 建立与调用 ..... 52
10.7 重入 ..... 52
10.8 字段初值 ..... 53
10.9 数据流图 ..... 53

第 11 章 不安全机制 ..... 54
11.1 安全区的边界 ..... 54
11.2 不安全区 ..... 54
11.3 指针操作 ..... 54
11.4 放宽的检查 ..... 55
11.5 审计友好 ..... 55
11.6 能力域与凭据 ..... 55
11.7 内联汇编 ..... 56
11.8 安全边界的重分类 ..... 57

第 12 章 宏与元编程 ..... 57
12.1 代码片段 ..... 57
12.2 宏的定义 ..... 58
12.3 引用与插值 ..... 58
12.4 编译期计算 ..... 58
12.5 使用边界 ..... 58
12.6 两种宏 ..... 59
12.7 声明注解 ..... 59
12.8 条件编译 ..... 59

第 13 章 所有权与移动 ..... 60
13.1 默认按值 ..... 60
13.2 显式移动 ..... 60
13.3 已移动状态 ..... 61
13.4 借用与转移的取舍 ..... 61
13.5 复制与克隆 ..... 61

第 14 章 界面元素块与图语法 ..... 61
14.1 元素块的写法 ..... 62
14.2 属性与绑定 ..... 62
14.3 事件 ..... 62
14.4 数据与界面 ..... 62
14.5 图语法 ..... 63
14.6 界面单元 ..... 63
14.7 元素与控制流 ..... 64
14.8 修饰与事件 ..... 64

第 15 章 工具链与语言服务 ..... 65
15.1 编译流程 ..... 65

15.2 命令行 ..... 65
15.3 诊断信息 ..... 66
15.4 格式化与静态检查 ..... 66
15.5 文档生成 ..... 66
15.6 交互式环境 ..... 67
15.7 语言服务 ..... 67

第 16 章 速查表 ..... 67
16.1 文件与角色 ..... 67
16.2 声明形态 ..... 68
16.3 参数约定 ..... 68
16.4 表达式记号 ..... 68
16.5 语句形态 ..... 69
16.6 常用内建 ..... 69
16.7 诊断编号 ..... 69
16.8 设计要点回顾 ..... 70
16.9 字符串与数字 ..... 70
16.10 运算符速览 ..... 70
16.11 语句速览 ..... 71
16.12 声明速览 ..... 71
16.13 形式与分工 ..... 72

第 17 章 生态协议与格式 ..... 73
17.1 td 与 tie:data ..... 73
17.2 zd：二进制序列化 ..... 74
17.3 tink 帧协议 ..... 76
17.4 tsha1 哈希家族 ..... 77
17.5 契约的协同 ..... 77

# 第 1 章 词法与记号

本章定义一个 tie 源文件的物理形态：如何组织、如何切分为记号、哪些记号有意义。词法层是其余一切章节的基础——语法分析只看到本章定义的记号流，而不关心源文件的字节细节。

## 1.1 源文件结构：头部与内容

一个 tie 源文件由两部分组成：头部声明本文件是什么，内容承载真正的代码或数据。两者职责分离，头部只占最前面几行。

```
type tie<logic>          // 头部：声明文件角色
                        // （其后为内容区）
func main() {
    println("hello")
}
```

头部是真正的语法行，而不再是以注释形式书写的编译器指令。头部必须位于文件最前若干行，连续排列（其间允许空行），直到第一个非头部行为止。

### 头部语法

头部由关键字 type、语言名 tie 与可选的尖括号子类型构成。子类型写在尖括号内；带参数的角色再用冒号给出参数，多个修饰用逗号分隔。

```
type tie                        // 裸形式：Type 角色本身
type tie<logic>                 // 带子类型的完整形式
type tie<db: vector, unsafe>    // 带参数与修饰
```

若文件未声明头部，按 logic 处理。文件名可以采用 xxx.角色.tie 的形式声明默认角色；当文件名与头部声明不一致时，以头部为准并给出一条警告。这样既允许用文件名快速辨别用途，又不让文件名凌驾于显式声明之上。

## 1.2 文件角色

角色决定编译器如何解析与生成该文件。它是文件的第一属性，决定了后续内容允许出现什么构造。

| 头部 | 角色与允许的内容 |
| --- | --- |
| type tie<logic> | 逻辑代码。可含函数、结构、模块级声明；可编译为可执行程序或静态库。默认角色。 |
| type tie<class> | 类库文件。与逻辑代码同为可执行语义，但只编译为静态库，不产生入口。 |
| type tie<tsc> | 脚本。顶层语句按书写顺序直接执行，不要求显式入口。 |
| type tie<data> | 纯数据。只允许数据声明与字面量，不允许函数体；可被其他文件导入作为常量源。 |
| type tie<ui> | 界面文件。声明界面单元树与交互契约，由编译器生成界面代码（见第 14 章）。 |
| type tie<db> | 数据库文件。声明表结构与查询，编译器生成建表与读写代码。 |
| type tie<port> | 接口文件。声明对外契约，不含实现。 |
| type tie<ir> | 中间表示文件。直接产出中间代码，不再经过后续优化与链接阶段。 |
| type tie<test> | 测试文件。只做检查与执行，不产出交付物。 |
| type tie<bench> | 基准文件。用于性能测量。 |
| type tie<zd> | 结构化二进制数据文件——数据格式的二进制变体（见第 9 章）。 |
| type tie<tsh> | 终端脚本文件，由终端脚本宿主执行（见第 15 章）。 |

### 修饰角色

修饰角色与基础角色并列书写，用于收紧或放宽该文件的语义边界。

| 修饰 | 作用 |
| --- | --- |
| unsafe | 整个文件处于不安全上下文。用于平台桥接与外部接口封装这类必须直接操作内存的模块。 |
| owned | 声明该文件的依赖归本项目所有（非外部依赖）。 |
| embedded | 目标为嵌入式场景，约束可用特性与运行时设施。 |

角色集是开放的：项目可以在自己的角色注册文件里声明自定义角色，使工程可以按领域细分文件的编译策略。

> 角色不是注释、不是标记，而是编译器分派解析与生成策略的依据。这正是头部必须在最前面、且被词法层正式识别的原因。

## 1.3 字符集与编码

源文件以 UTF-8 编码。字符串内部同样以 UTF-8 存储，但语言层面区分两个尺度：

- s.len() / s.char(i) —— 以码点为单位。中文、emoji 都算一个。
- len(s) —— 以字节为单位。一个汉字占 3 字节。

遍历文本一律使用码点尺度；按字节遍历会把多字节字符切碎。这一约定贯穿标准库的文本处理函数。

```
var s = "你好"
s.len()       // 2    码点
len(s)          // 6    字节
s.char(0)   // "你"
```

## 1.4 注释

注释有三种形态，前两种在词法阶段被丢弃，不进入记号流；第三种会附着到声明上。

```
// 行注释：自 // 起至本行结束
/* 块注释：可跨行 */
/// 文档注释：附着到紧随其后的声明
```

文档注释可被工具提取，用于生成接口文档。它比普通注释有更强的归属语义：紧接在某个声明之前，就属于那个声明；写在函数体内部则退化为普通注释。

## 1.5 标识符

标识符由字母、下划线或 Unicode 字母开头，其后可接字母、数字与下划线。语言接受 Unicode 标识符，中文可以直接用作标识符。

```
count, _tmp, 用户数, 结果1
```

约定：类型名以大写字母开头，函数与变量以小写字母或下划线开头。这是约定而非强制，编译器不做检查，但工具链会据此提供提示。

## 1.6 关键字

关键字是保留字，不可用作标识符。它们按用途分组如下。

| 用途 | 关键字 |
|---|---|
| 声明 | var const func struct enum namespace port interface impl import as pub using extern macro alias actor extends |
| 类型 | i8 i16 i32 i64 u8 u16 u32 u64 i128 u128 f32 f64 bool trit char string void code table map num text misc any |
| 语句 | if else while for in switch case default when break continue return yield goto |
| 修饰与更新 | ref immut move deep shared inline repr unsafe const async with reentrant |
| 表达式 | is await |
| 字面量 | true false zero |
| 并发与故障 | panic |
| 内联汇编 | asm |

> 关键字表是封闭的：新关键字只在语言版本升级时加入。语言另有一批只在特定位置被识别的词（见 1.13），它们不是保留字，但同样不宜用作标识符。

## 1.7 数值字面量

整数支持五种进制，允许用下划线分组以提高可读性。小数与指数形式构成浮点字面量。字面量可带类型后缀，显式指定其类型。

| 形态 | 含义 |
| --- | --- |
| 42 | 十进制整数，默认类型 i64 |
| 0xFF / 0xff | 十六进制 |
| 0b1011 | 二进制 |
| 0o755 | 八进制 |
| 0t1T0 | 平衡三进制（数字 0 / 1 / T，T 表示 -1） |
| 1_000_000 | 下划线分组，等价于 1000000 |
| 0xCA_FE | 分组在各进制中均可 |
| 3.14 / 1.5e3 | 浮点，默认类型 f64 |
| 1.2f32 / 42i32 / 0x80u16 | 带类型后缀，显式指定类型 |
| -1t / 0t / 1t | 三值字面量（等价于 false / zero / true） |

## 1.8 布尔与三值字面量

true 与 false 构成布尔类型。语言另有一个三值类型 trit，取值为正、零、负，其字面量复用 true / zero / false，在 trit 上下文中适配为三值。

```
var b: bool = true
var p: trit = true     // 正
var z: trit = zero     // 零
var n: trit = false    // 负
```

## 1.9 字符与字符串字面量

字符字面量以单引号界定，字符串字面量以双引号界定。字符串有四种形态，各有其适用场合。四种形态里只有插值字符串会求值花括号内的表达式，而它必须以 h 前缀显式标出——花括号是否被当作替换点，看字面量的头一个字符就知道。

| 形态 | 适用场合 |
| --- | --- |
| 普通字符串 "..." | 转义生效；花括号原样保留。日常使用。 |
| 原始字符串 r"..." | 转义不生效、无插值。写路径、正则、含反斜杠的内容用它，省去双重转义。 |
| 多行字符串 """..""" | 可跨行书写，按闭合引号的缩进剥离每行前缀。写 SQL、JSON、模板片段用它，不破坏源码缩进。 |
| 插值字符串 h"...{e}..." | 花括号内表达式求值后转为文本拼入；跨行时写作 h"""..."""，同样带插值。 |

```
var c: char = 'A'
var esc = "tab:\t quote:\" backslash:\\"
var raw = r"C:\path\to\file"        // 无转义，原样保留
var plain = "共 {n} 项"             // 花括号原样保留
var interp = h"共 {n} 项"           // 花括号内求值后拼入
```

多行字符串按闭合引号所在行的缩进作基准，逐行剥离该前缀。这样源码可以在任何缩进层级书写 SQL 或 JSON 片段，而字符串内容本身保持左对齐。

插值是显式开启的行为，不是普通字符串的隐含语义。含花括号的内容——JSON、模板片段、正则——因此可以照原样书写，不必担心被意外求值；确实要把运行时值拼进去时，才在引号前写上 h。h 不能与形态前缀 r 组合：原始字符串的用途正是「一个记号也不解释」，而 h 要求解释花括号。

插值字符串里要写出字面的花括号时，写两个花括号代表一个；不带 h 的字符串没有这条规则，花括号即字面量。

插值点可以携带格式说明符，写在表达式之后、以冒号分隔：花括号内写作「表达式:规格」。规格自左向右依次是对齐与填充、正负号、宽度、精度与类型，各段均可省略。有了它，数值的定点、补零、进制转换这类日常格式化就在插值里一并完成，不再需要单独的格式化函数族。

| 规格段 | 含义 |
| --- | --- |
| 对齐与填充 | < 左对齐、^ 居中、> 右对齐；可在其前写一个填充字符，默认空格。 |
| 符号 | + 表示非负数也显式写出加号。 |
| 宽度 | 十进制整数，最小字符数，不足按对齐补齐。 |
| 精度 | .n：浮点限定小数位，字符串限定最大长度。 |
| 类型 | b 二进制、o 八进制、x 十六进制、e 科学计数、f 定点、% 百分比。 |

```
h"{x:08.2f}"          // 定点两位小数，宽八，不足补零
h"{id:>6x}"           // 十六进制，右对齐，宽六
h"{ratio:%}"          // 百分比
h"{name:^10}"         // 字符串居中，宽十
```

## 1.10 自动分号

tie 不要求每条语句以分号结尾。编译器在词法阶段按规则补全语句结束，这一机制称为自动分号插入。

- 当一行结束后，若下一个记号不能延续当前语句，则在此处插入语句结束。
- 当一行以逗号、二元运算符、左括号等「明显未结束」的记号结尾时，不插入。
- 显式书写分号始终合法，可与自动补全混用。
- 括号深度只统计圆括号与方括号；花括号是块边界，不计入。
- 注释、字符串、插值块内部不参与该判定。

这条规则让代码更接近日常书写的文本，同时保持分析的确定性——判定只看记号，不依赖缩进或换行风格。

## 1.11 行续行

行尾写反斜杠表示「本行未结束」，下一行继续同一语句。它用于把长表达式拆成多行，且不依赖运算符的位置。

```
var total = first + \
    second + \
    third
```

自动分号与续行是互补的两条规则：前者在语句该结束时补上结束符，后者明确表示此处不该结束。需要跨行书写的长表达式应当显式续行，而不依赖「行尾恰好是运算符」这一间接判据。

## 1.12 运算符记号

下表汇总全部运算符记号；含义与结合性见第 3 章，第 8 章给出记号一览与优先级表。

| 记号 | 用途 |
|---|---|
| + - * / % | 算术；取余 |
| \ %% | 整除；成对取余 |
| ** **? | 幂；带检查的幂 |
| +? -? *? <<? | 带检查的加、减、乘与移位 |
| == != < > <= >= | 比较（支持链式比较） |
| && \|\| ! | 逻辑（对三值为三值逻辑） |
| & \| ^ << >> ~ | 位运算 |
| = += -= *= /= %= %%= | 赋值与复合赋值 |
| \= &= \|= ^= <<= >>= | 其余复合赋值 |
| ++ -- | 自增与自减 |
| -> | 数据流管道；返回位；短闭包；分支臂 |
| <- | 数据流反向赋值；反向解构 |
| in  not in | 成员判定与取反 |
| ?: | 安全默认值 |
| ? ?. ?[ | 存在判定与可空链访问 |
| \ | 解引用（不安全上下文） |
| .. ..= | 范围（左闭右开 / 左闭右闭） |
| ... | 变长形参、实参展开、解构剩余 |
| @ # $ ` | 元素标记与注解、属性与标签、宏插值、准引用 |
| with | 不可变更新 |
| -\| <-\| ->x ~x ->< | 图的边与节点操作（不安全上下文） |

## 1.13 上下文关键字

有一类词只在特定位置被识别为语法成分，其他位置是普通标识符。它们不计入保留字，但书写时不应作为变量名使用。

| 词 | 识别位置 |
|---|---|
| fn | 函数类型与函数修饰位置 |
| let | 条件解构位置 |
| not | 与 in 组合成成员取反 |
| try | 错误聚合块位置 |
| defer | 作用域收尾块位置 |
| guard | 前置条件位置 |
| run | 建立并发执行体的位置 |
| self | 接收者位置 |
| out / inout / target | 内联汇编操作数位置 |
| array | 定长数组类型位置 |
| r | 紧跟引号时表示原始字符串 |
| h | 紧跟引号时表示插值字符串 |
| type | 文件头部声明角色 |

> 把这批词与保留字分开列出，是因为二者的约束强度不同：保留字是硬约束（用作标识符即报错），上下文关键字是命名建议（用作标识符不报错，但在相应位置会产生歧义）。

# 第 2 章 类型系统

tie 是静态类型语言：每个表达式在编译期有确定的类型，类型错误在编译期报告，不进入运行期。类型系统同时承担两件事——描述数据的形状，以及表达访问权限（谁可以写、谁只能读、谁拥有所有权）。后者见第 5 章与第 13 章。

类型分为若干族：基本类型、复合类型（表、元组、映射）、聚合类型（结构、枚举）、函数与接口类型、以及两种特殊类型（宽类型与动态类型）。

## 2.1 基本类型

基本类型直接对应机器表示，没有隐藏的分配与间接层。

| 类型 | 含义 |
| --- | --- |
| i8 i16 i32 i64 | 有符号整数，宽度以位计 |
| u8 u16 u32 u64 | 无符号整数 |
| i128 u128 | 128 位整数 |
| f32 f64 | 浮点数 |
| bool | 布尔，取值 true / false |
| trit | 平衡三进制，取值正 / 零 / 负（见 2.2） |
| char | 单个字符，按码点计 |
| string | 不可变文本，UTF-8 存储 |
| void | 无值，仅作函数返回类型 |
| code | 编译期代码片段，无运行期实体（见第 12 章） |

整数默认类型为 i64，浮点默认类型为 f64。字面量在需要的上下文中会按目标类型调整。

## 2.2 平衡三进制 trit

trit 是一个三值类型，取值正、零、负。它用于三路比较、三值逻辑与平衡三进制数论。与布尔相比，它多了一个「未知」状态，因此比两者择一更贴近真实判定。

```
var cmp: trit = compare(a, b)   // 正 表示 a>b，零 表示相等，负 表示 a<b
if cmp == zero {
    // 相等分支
}
```

三值逻辑的运算规则与布尔一致地推广：与运算取小、或运算取大、非运算取反。混合 trit 与整数的运算按提升规则转为整数，除法与取余对 trit 不开放。

## 2.3 宽类型

宽类型是一组类型的类别名，用于在书写时省略具体宽度，或在多种数值类型间统一处理。它只存在于编译期：变量在参与运算时使用推导出的具体类型。

| 关键词 | 覆盖范围 |
| --- | --- |
| num | 全部数值类型：整数与浮点 |
| text | 文本类型：string 与 char |
| misc | 其余类型：bool、void、code 等 |

```
var a: num = 42        // 推导为 i64
var b: num = 3.14      // 推导为 f64
var s: text = "hi"     // 推导为 string
```

宽类型最常见的用途是编写同时接受多种数值宽度的函数，例如统计与格式化工具。

## 2.4 表类型

表是动态长度的同质序列，元素类型写在尖括号内。创建表不需要专门的构造器：字面量即构造，空表写作方括号对。

```
var fixed: table<i64> = [1, 2, 3]        // 字面量表
var dyn: table<i64> = []                 // 空表，元素类型由标注确定
var grown = [1, 2]                       // 无标注：由字面量推导
dyn <- 1                                 // 追加
dyn <- 2 <- 30                           // 追加可链式
```

空表的元素类型从两处确定：有类型标注时取标注；无标注时取首次追加或比较的元素类型，若无从确定则报编译错。需要显式指明时，泛型构造函数 table_new<T>() 与空字面量等价。

表可嵌套，形成多维结构。遍历使用 for-in，长度用 len。

```
var grid: table<table<i64>> = [[1, 2], [3, 4]]
for row in grid {
    for cell in row {
        println(to_string(cell))
    }
}
```

## 2.5 映射类型

映射是以键索引的集合，键与值都有类型。键可以是文本、整数或枚举。空映射写作 [:]，与空表相区分。

```
var age: map<i64> = ["alice": 3, "bob": 5]
var empty: map<i64> = [:]
age["carol"] = 7
if "alice" in age {
    println(age["alice"] -> to_string)
}
```

映射的字面量语法与表一致，区别在于元素写成「键: 值」。当键为字符串时，可用点号访问其值，这在表达结构化数据时更易读。

## 2.6 元组类型

元组把若干个类型组合成一个值，位置固定、类型可不同。元组用于返回多个结果，或表达「这几个值一起才有意义」的场合。

```
func min_max(xs: table<i64>) -> (lo: i64, hi: i64) {
    // ...
    return (1, 9)
}

var (lo, hi) = min_max([3, 1, 9])
```

元组可命名分量，命名同时作为文档与解构依据。也可不命名，此时用位置访问。

## 2.7 类型标注与推导

tie 的立场是：类型系统是静态的，标注是可选的。类型错误永远是编译错，不推迟到运行期；用户可自由选择写法--动态语言写法或静态语言写法，省略的标注会在编译期推导出来，推导不出来会报错。省略标注时的手感接近动态语言，安全性与运行期效率不变。

```
var n: i64 = 0          // 显式标注
var m = 0               // 推导为 i64
var s = "text"          // 推导为 string
var rows: table<Entity> = []   // 容器空字面量：标注给出元素类型
```

### 标注可省略的位置

- 局部变量：从初始化表达式推导。
- 结构字段：带初值时从初值推导。
- 函数返回类型：从函数体推导，等号体与块体均可。
- 闭包与短闭包参数：从上下文的函数类型推导。
- 模块内（非 pub）函数的参数与返回类型：从定义与调用点联合推导。

### 必须标注的位置

pub 声明的函数与结构字段必须写出参数与返回类型。公开边界是契约所在，类型在这里起文档与检查的双重作用；边界之内才是推导的领地。这条划分让库作者在边界处想清楚类型，让使用方与内部实现写得随意。

```
pub func clamp(v: i64, lo: i64, hi: i64) -> i64 { ... }

func help(a, b) { return a + b }   // 模块内：参数与返回类型全省略
```

模块内函数的类型仍然完全确定：推导发生在编译期，每个名字都有精确的静态类型，调用错了同样编译报错。省略标注省的是书写，不是检查。

## 2.8 泛型

泛型让类型与算法解耦。类型参数写在尖括号中，编译期为每个具体类型生成一份实现，因此运行期没有额外开销，也没有虚表跳转。

```
func max_of<T>(a: T, b: T) -> T {
    if a > b { return a }
    return b
}

struct Box<T> {
    var value: T
}
```

类型参数可带约束，约束由接口描述（见第 7 章）。约束在编译期检查，不满足即报错。

```
func render_all<T: Drawable>(items: table<T>) {
    for it in items {
        it.draw(0)
    }
}
```

## 2.9 枚举

枚举是标签联合：一个值要么是这个标签、要么是那个标签，并可按标签携带数据。它用于表达「几种情况之一」的数据，比用整数代号更安全——每个分支都被检查。

```
enum Shape {
    Circle(r: f64)
    Rect(w: f64, h: f64)
}

func area(s: Shape) -> f64 {
    switch s {
        case Shape.Circle(r):
            return 3.14159 * r * r
        case Shape.Rect(w, h):
            return w * h
    }
}
```

枚举也可以做通用型参数，用于表达可能失败或可能缺失的结果，这是第 8 章错误处理的基础。

## 2.10 动态类型

any 表示「任意类型」。它在需要容纳异质数据的场合使用，例如配置表、序列化中间结果。任何值都可以赋给 any，取用时需要显式转换。

```
var v: any = 42
v = "text"
if v is i64 {
    var n = as_i64(v)
}
```

> 静态类型是默认选择。any 把类型错误的发现从编译期推迟到运行期，因此只在确实需要异质容器时使用，而不是作为绕过类型检查的手段。

## 2.11 类型转换

数值类型之间的转换是显式的。窄化转换可能丢失信息，语言要求书写者明确表达意图。

```
var a: i64 = 1000
var b = to_i32(a)        // 窄化：显式
var c: f64 = to_f64(a)   // 整数到浮点
var d = to_string(a)     // 到文本
```

指针与整数之间的转换只在 unsafe 上下文中允许（见第 11 章）。聚合类型与动态类型之间的转换由各自的构造与匹配机制承担。

转换函数按三族命名，职责不重叠：to_ 系做数值转换与任意值转文本，tp_ 系做可预期的文本解析，as_ 系从动态类型解包。

| 函数族 | 语义 |
| --- | --- |
| to_i8 … to_u128 / to_f32 / to_f64 | 数值类型之间的转换；窄化必须显式写出。 |
| to_string(v) | 任意值转文本。 |
| to_i64(s) / to_f64(s) | 文本解析为数值；解析失败触发内建检查，适用于「解析不了就是程序错误」的场合。 |
| tp_i64(s) / tp_f64(s) | 解析为 Result<i64, string> / Result<f64, string>；失败可预期，由调用方分支处理。 |
| as_i64 / as_f64 / as_string / as_bool | 从 any 解包；实际类型不符即报错。 |

```
var n = to_i64("4096")          // 文本解析，失败即内建检查
var r = tp_i64(s)               // 可预期失败：Result<i64, string>
var m = as_f64(boxed)           // 从 any 解包
```

## 2.12 低层与特殊类型

以下几类类型用于系统编程、并发与资源管理，它们多数只在不安全上下文或特定场景中出现。

| 类型 | 说明 |
| --- | --- |
| ptr<T> | 裸指针。指向 T 的地址。类型与其值的声明、比较、传递在安全代码中即可用；解引用与算术须不安全上下文（见 11.8）。 |
| slice<T> | 切片。指向一段连续元素视图，共享底层数据而不复制。 |
| atomic<T> | 原子类型。提供原子读写与比较交换，用于跨执行流的无锁同步。 |
| guard<cap> | 凭据。只能转移、不能复制的许可票据，cap 取值见第 11 章。 |
| array<T, N> | 定长数组。长度是类型的一部分，在编译期已知。 |
| fn(A, B) -> R | 函数类型。可作为参数、返回值、字段与容器元素。 |

## 2.13 定长数组

定长数组的长度写在类型里，因此长度不同的数组是不同类型。它适合表达固定规模的数据——向量、矩阵行、环形缓冲。

```
var v: array<f64, 3> = array_init(0.0)
var m: array<array<f64, 3>, 3> = array_init(v)
```

长度可以是整数字面量，也可以是常量名或泛型值参数。长度参与类型检查，因此当下标为常量时，越界在编译期即可发现。

## 2.14 函数类型

函数类型用箭头书写，与函数定义的返回位写法一致。任何函数值都有确定的函数类型，包括具名函数、闭包与短闭包。

```
var cb: fn(i64) -> i64 = double
var hook: fn(string) -> void
func apply(g: fn(i64) -> i64, v: i64) -> i64 { return g(v) }
```

返回类型可省略——从函数体推导（见 2.7）；pub 声明必须显式写出。无返回值的函数写 void。函数类型可以嵌套，返回函数的函数写作 fn(A) -> fn(B) -> R。

## 2.15 行池

当表的元素是结构时，同一种写法具有另一种语义：它成为按标识符索引的行池。行池的每行有稳定标识，行可以稀疏存在，读取未写过的行得到默认值。

```
var rows: table<Entity> = []
rows[id] = e            // 写入该标识的行
var r = rows[id]        // 读取；未写过的行得到默认值
len(rows)               // 最大已写标识加一
```

行池的价值在于按标识直接定位：不必在表中线性查找，也不必维持标识与下标的一致。它在界面的列表、场景的实体、表格的稀疏数据等处反复出现。行遵循值语义——读出的是副本，写回需要显式赋值。

## 2.16 精确数值类型

浮点无法精确表示十进制小数，这在账目、计量与规格表示中不可接受。语言提供两个精确数值类型：十进制与任意精度整数。

| 类型 | 语义 |
| --- | --- |
| 十进制 | 以 符号、系数、十的幂 三元表示。加减乘恒精确；除法与开方在能精确时精确，否则按显式给定的精度与舍入方式处理。无负零、无无穷、无非数。 |
| 大整数 | 任意精度整数。用于超出机器字长的计数、密码学与数论计算；进制转换覆盖二至四十八进制。 |

```
var price = 0.1d + 0.2d        // 精确等于 0.3，而非 0.30000000000000004
var big = big_parse("12345678901234567890123456789O")
var q = div_ctx(a, b, 10, half_even)   // 除法显式给定精度与舍入
```

零偏差是这两个类型契约的核心：凡是数学上精确的结果，实现就必须精确给出，不允许以「足够接近」代替。舍入只发生在显式要求舍入的调用中，不存在隐式舍入与全局设置。

## 2.17 类型别名

别名给类型起第二个名字，语义上与原类型完全相同——它不是新类型，不产生任何转换边界。别名用于压缩冗长类型、给领域概念以恰当名称。

```
alias Matrix = table<f64>
alias Pair<T> = (T, T)
alias Handler = fn(Request) -> Response
```

诊断信息会同时给出原名与别名，因此别名不会让错误信息显得陌生。

## 2.18 泛型的值参数

类型参数之外，泛型还可以接受值参数：在编译期确定的整数或类型。它用于表达规模、容量、精度这类应当是类型一部分的量。

```
func zeros<N: i64>() -> array<f64, N> { return array_init(0.0) }
struct Buffer<CAP: i64 = 1280> { var data: array<u8, CAP> }
var b = Buffer<512>()          // 容量成为类型的一部分
```

值参数有默认值时可以省略。把规模放进类型的好处是：规模不匹配在编译期即报错，而不是在运行期越界。

## 2.19 字符串处理

字符串的内建操作以 str_ 前缀的自由函数提供，与 str_len、str_char 同族。全部操作以码点为尺度（见 1.3），不按字节切碎文本。

| 函数 | 语义 |
| --- | --- |
| s.split(sep) | 按分隔符拆分为表<string>；分隔符不保留。 |
| parts.join(sep) | 以分隔符拼接表<string>。 |
| s.contains(sub) | 是否包含子串。 |
| s.starts(pre) / s.ends(suf) | 前缀与后缀判定。 |
| s.replace(old, new) | 替换全部出现的子串。 |
| s.trim() | 去除首尾空白。 |
| s.reverse() | 按码点逆置。 |
| s.upper() / s.lower() | 大小写转换（仅 ASCII 字母，其余原样）。 |

```
var fields = "a,b,c".split(",")     // ["a", "b", "c"]
var line = fields.join(" | ")        // "a | b | c"
if name.starts("test_") { }
```

> 需要把数值拼进文本时，优先用插值字符串与格式说明符（见 1.9）；str_ 系承担的是结构化文本操作——拆分、拼接、查找、修剪——而非格式化。

## 2.20 集合操作

表与映射的内建操作同样是自由函数。原地生效的操作返回自身，因此可以直接接入管道；产出新值的操作不改原表。

| 函数 | 语义 |
| --- | --- |
| seq_sort(t) | 升序排序，元素需可比较；原地生效并返回自身。 |
| seq_sort_by(cmp, t) | 以比较函数排序（cmp(a,b) =「a 应先于 b」）；原地生效并返回自身。 |
| seq_reverse(t) | 逆置；原地生效并返回自身。 |
| seq_distinct(t) | 保序去重，返回新表。 |
| tbl_cat(a, b) | 拼接两表，返回新表。 |
| seq_group_by(key, t) | 按键函数（返回字符串键）分组，返回 map<table<T>>。 |

```
var top = scores -> seq_sort() -> seq_reverse()
var rows = seq_group_by(func(o: Order) -> string { return o.city }, orders)
```

排序是稳定的：相等元素的相对次序在排序后保持不变。这让多重排序可以先按次关键字排序、再按主关键字排序地组合出来。

# 第 3 章 运算符与表达式

表达式由运算符与操作数构成。本章按优先级自高到低说明各运算符的形态与语义，并单列 tie 特有的若干记号——它们的共同目标是让常见意图有专有的书写形式，而不是靠函数调用或状态变量绕行。

## 3.1 优先级与结合性

优先级自高到低如下表。同一行的运算符优先级相同，按结合性方向结合。括号可覆盖默认优先级。

| 级 | 运算符 | 结合性 |
|---|---|---|
| 1 | 后缀 [ ] . :: ( ) ?. ?[ | 左 |
| 2 | 一元 - ! ~ ++ -- \ | 右 |
| 3 | ** **? | 右 |
| 4 | * / % \ %% *? | 左 |
| 5 | + - +? -? | 左 |
| 6 | << >> | 左 |
| 7 | < > <= >=（可链式） | 左 |
| 8 | in not in | 左 |
| 9 | == != | 左 |
| 10 | & | 左 |
| 11 | ^ | 左 |
| 12 | \| | 左 |
| 13 | && | 左 |
| 14 | \|\| | 左 |
| 15 | ?: 条件默认 | 右 |
| 16 | -> <-（数据流） | 左 |
| 17 | with 不可变更新 | 左 |
| 18 | .. ..= 区间 | — |

> 两条容易记错的相对关系：幂高于乘除，且优先与左侧一元运算结合（故 -2 ** 2 等于 4）；成员判定比相等比较更紧（故 x in xs == false 等于 (x in xs) == false）。

## 3.2 算术运算

算术运算符对整数与浮点生效，规则与通常预期一致。整数除零在运行期报错，语言不做静默截断。

```
var a = 7 + 3      // 10
var b = 7 / 3      // 2      整数除法截断
var c = 7.0 / 3.0  // 2.333  浮点除法
var d = 7 % 3      // 1      取余
```

整数运算在溢出时按位宽回绕。需要检出溢出时使用带检查的形式（见 3.5）。

## 3.3 整除与成对取余

整除与取余是一对记号：整除给出向下取整的商，取余给出与之配对的余数。二者成对使用，保证商与余数自洽——即 商乘除数加余数恒等于被除数。这是普通乘除取余在负数上无法满足的性质。

```
var q = 7 \ 3      // 2      整除（向下取整）
var m = 7 %% 3     // 1      成对取余
q = -7 \ 3         // -3
m = -7 %% 3        // 2      自洽：-3 * 3 + 2 == -7
q \= 2             // 整除复合赋值
```

> 整除曾用两个斜杠书写，因与行注释冲突而改为反斜杠。反斜杠同时用于解引用（见 3.14），但两者位置不同：作二元运算符时是整除，作一元前缀时是解引用。

## 3.4 幂运算

幂运算写作两个星号，右结合，因此连续幂按从右往左分组。

```
var p = 2 ** 10        // 1024
var q = 2 ** 3 ** 2    // 512   右结合，即 2 ** (3 ** 2)
```

带检查的幂写作 `**?`，在结果超出该类型表示范围时报告错误，而不是回绕。数值敏感的计算应使用这一形式。

## 3.5 带检查的运算

加、减、乘、幂与移位各有一个带检查的变体，运算符后接问号。溢出或越界时不再回绕，而是触发可捕获的错误。

```
var s = a +? b      // 溢出即报错
var d = a -? b
var m = a *? b
var p = a **? b
var sh = a <<? b    // 移位量越界即报错
```

同一能力另有函数形态，便于在不希望中断流程处使用：这类函数返回结果与「是否溢出」两个值，由调用方决定如何处置。

```
var (sum, ok) = checked_add(a, b)
if !ok {
    println("溢出，改用更宽的类型")
}
// 同族：checked_sub / checked_mul / checked_div / checked_shl / checked_shr / checked_neg
```

> 两套机制并存是有意设计：运算符形态用于「溢出即异常」的情形，写起来最短；函数形态用于「溢出可预期、需要分支处理」的情形。二者的判定标准完全相同，因此同一表达式在两种写法下结论一致。

## 3.6 比较运算

比较运算符返回布尔值。对文本按码点顺序比较；对元组按分量依次比较。

连续比较可以链式书写，等价于两两比较再用与运算连接——这使「落在区间内」这类判断读起来与数学写法一致。

```
1 == 1          // true
"abc" < "abd"    // true
if 0 <= x < 10 {    // 等价于 (0 <= x) && (x < 10)
    println("在区间内")
}
```

相等比较要求两侧类型相容。语言不为不同类型之间定义隐式相等——这避免了整数与浮点混比时的意外结果。

## 3.7 逻辑运算与三值逻辑

逻辑运算符对布尔生效，并短路求值：与运算在左操作数为假时不再求右操作数，或运算在左操作数为真时不再求右操作数。

当操作数是三值类型时，同一组运算符按三值逻辑解释：与取较小值、或取较大值、非运算取反。由于三值包含「未知」，与运算中只要有否即为否，或运算中只要有真即为真，未知只在无法决定时才保留。

```
var t: trit = unknown_result()
if t && positive_signal() {
    // 仅当两者皆正
}
```

这组规则的作用是把「未知」当作一等值传播，而不是让它退化成真假之一。三路比较与三态判定都直接受益。

## 3.8 位运算

位运算符作用于整数，移位运算的右操作数是移位数。

```
var m = 0b1100 & 0b1010   // 0b1000
var n = 1 << 4            // 16
var x = ~0                // 按位取反
```

## 3.9 成员判定

成员判定用于回答「这个键在不在这个结构里」。它是布尔表达式的一部分，可直接用于条件。判定对象可以是映射（判键）、表（判元素）、文本（判子串）。

```
if "alice" in age {
    println("已知")
}
if "dave" not in age {
    println("未见")
}
```

取反形式 not in 是一个整体记号，不写成 !(x in y)——后者在长条件里容易读错。判定成本随容器而异：映射判键是常数时间，表判元素与文本判子串是线性时间。

## 3.10 加法的语义扩展

加号在数值相加之外还有三种扩展，都遵循值语义：产生新值，不修改操作数。

| 形态 | 语义 |
| --- | --- |
| 文本 + 标量 | 标量转为文本后拼接。左右顺序均可。 |
| 表 + 表 | 两表元素依序拼接为新表（全量复制）。 |
| 映射 + 映射 | 合并为新映射；键冲突时右侧覆盖左侧。 |

```
var msg = "n = " + n            // 文本拼接，免手写转换
var all = xs + ys               // 表拼接
var cfg = base + override       // 映射合并，右侧优先
```

### 重复与集合运算

乘号在文本与表上表示重复，四个位运算记号在容器上按集合语义重载——与整数位运算共享记号，按操作数类型分派，机制与加法的扩展相同。四则运算符取的是「值的集合」语义：按值判定相同，结果保序。

```
var sep = "-" * 20              // 文本重复
var grid3 = [0] * 3             // [0, 0, 0]
var both = a | b                // 并：a 与 b 的全部值
var core = a & b                // 交：共同拥有
var diff = a - b                // 差：a 有 b 无
var flip = a ^ b                // 对称差：仅一方拥有
```

这组记号只对表与映射定义，触发条件是操作数为容器。需要区分「重复三行的表」与「三个元素」这类边界时，用切片与比较表达，运算符不承担歧义语义。

## 3.11 数据流箭头

箭头是 tie 表达数据流向的主要手段。它把「值经过什么」写成一条自左向右的链，使变换序列的阅读顺序与执行顺序一致。

### 管道

管道把左侧的值作为右侧调用的输入。嵌套调用与管道写法等价：

```
// 嵌套写法
println(to_string(len(input.trim())))

// 管道写法
input -> trim -> len -> to_string -> println
```

当调用还需要额外实参时，被管道的值作为最后一个实参传入。

```
10 -> add2(5)       // 等价于 add2(5, 10)
```

### 管道进入容器与字段

箭头后可以紧跟下标或字段，表示把值送进容器或字段读取。

```
var first = t -> [0]        // 等价于 t[0]
var name = user -> .name    // 等价于 user.name
```

### 管道接运算符

箭头后可以直接跟一个二元运算符，表示在该值上施加那个运算。它使「在链中间做一次小变换」不必打断链条。

```
var grown = t -> + [x]       // 等价于 t + [x]
var hit = key -> in table    // 等价于 key in table
```

### 块管道

箭头后跟一个花括号块，块内首个箭头之前的标识符即为绑定名，块内最后一个表达式的值即结果。它用于在链条中间插入一段需要临时变量的计算。

```
var r = n -> {
    v -> v * 2 + 1          // v 是绑定名
}
```

### 条件管道

箭头也可以进入条件表达式，用于在链条中分叉。

```
var label = score -> score >= 60 ? "及格" : "不及格"
```

### 反向箭头

反向箭头把右侧的值写入左侧目标，是管道在赋值方向的镜像。它同样支持下标与字段目标。

```
x <- 42             // 等价于 x = 42
arr[0] <- 9         // 等价于 arr[0] = 9
user.name <- "ann"  // 等价于 user.name = "ann"
```

左侧目标是容器本身时，反向箭头表示追加：值流入容器尾部。追加返回容器自身，因此可以链式连写。下标写入仍用等号——「把 v 放进 k 位」与「把 v 追加进 t」是两个意图，各用各的记号。

```
t <- 1 <- 2 <- 30   // 依次追加，等价于三句 t <- v
t[0] = 9            // 下标写入仍用等号
t -> table_pop()    // 取出尾部元素：数据送进操作（表不设点，见下）
```

### 点调用与箭头的分工

点号专用于「从属关系」：点号右侧的名字住在左侧实体里面——结构的字段与方法、枚举的变体、元组的元素、命名空间与文件的成员。纯数据值（表、字符串等）没有成员可住，因此表与字符串上不写点；它们的操作送进外部函数（箭头），或使用专门的记号——追加用 `<-`、下标用 `[]`。判据是「这个名住在哪里」：住在左侧实体里的用点，住在外面的用箭头。

```
p.x                 // 字段：住在结构里
p.move_by(1.0, 1.0) // 方法：定义在结构的同名命名空间
fs.exists(path)     // 命名空间成员
lib.c()             // 文件顶层成员（函数；文件词干限定）
lib.K               // 文件顶层常量/变量（值成员）
Color.Red           // 枚举变体
t -> table_pop()    // 数据送进操作：箭头（数据作末参）
xs -> validate(t)   // 送进外部函数：箭头
```

二者可以自由衔接：需要限定从属时用点，需要把值送进变换时用箭头。数据本身的就地写入另有专门

记号（追加 `<-`、下标 `=`），三种写法各司其职。

### 箭头用于解构

反向箭头配合元组形态即解构赋值。

```
(lo, hi) <- min_max(xs)
(q, r) <- divmod(7, 3)
```

### 与短闭包的区分

形如 标识符 -> 表达式 的写法在两种情况下含义不同：在「要求函数值」的位置是短闭包，其余位置是管道。判据是上下文，而不是符号表。

```
var f: fn(i64) -> i64 = t -> t + 1   // 声明位：短闭包
f = t -> t * 2                       // 赋值位（目标为函数类型）：短闭包
var v = src -> f                     // 其他位置：管道
```

> 箭头承担五种职责（管道、函数返回位、分支臂、短闭包、解构目标），但它们出现的位置互不重叠，因此不会产生歧义。语言不使用第二个箭头记号来表达这些含义——一个记号、五种位置，记忆负担最小。

## 3.12 不可变更新

with 记号为结构或映射生成「复制并覆盖指定成员」的新值。未提及的成员原样保留，且在可能时共享底层数据而不做全量复制。

```
var p2 = p with { x: 1 }                 // 结构：复制并覆盖字段
var t2 = t1 with { k: v }                // 映射：不可变更新
var t3 = t1 with { a: 1, b: 2 }          // 一次多键
var t4 = t1 with { a: 1 } with { b: 2 }  // 连续更新
```

这一记号的价值在于把「基于旧值造新值」写成一行，并明确表达原值不变。没有它时，同样的意图要写若干行赋值，读者还需自行确认原值是否被改动。

## 3.13 可空链

可空值写作 Option<T>（Some 持值、None 表示缺失，见 8.1）。它有三种处理记号：判定是否存在、给出默认值、沿链继续访问。三者配合，使「取值可能为空」这条常见路径不必层层写分支。

| 记号 | 语义 |
| --- | --- |
| expr ? | 取出 Option 内的值；为 None 则立即从当前函数返回（错误向上传播）。 |
| expr ?: 默认值 | 取出 Option 内的值；为 None 则用右侧表达式。右侧只在缺失时求值。 |
| opt ?. 字段 | 在可空值上取字段；为 None 则整体为 None，不继续求值。 |
| opt ?. 方法() | 在可空值上调用方法；同上。 |
| opt ?[ 下标 ] | 在可空值上取下标；为 None 或越界则整体为 None。 |

```
var port = opt_port ?: 8080              // None 则取默认
var city = user?.address?.city ?: "未填"  // 沿链安全访问
var x = parse(s) ?                       // 失败则向上传播
```

可空链的语义是缺失短路：链上任一环为 None，后续访问不再发生，整个表达式求值为 None。这既免去了对空值的逐层判断，也避免了「先判断再访问」之间可能出现的状态变化。

这些记号只作用于 Option。裸表与裸映射的下标不产生缺失：越界或不存在的键按 8.5 由内建检查报错。需要「越界即缺失」时，先把值包进 Option（Some），再用 opt?[i]。

## 3.14 解引用

反斜杠作为一元前缀时是解引用记号，作用于指针与引用，取出其所指的值。它只在不安全上下文中可用。

```
unsafe {
    var p = alloc(8)
    var v = as_i64(\p)      // 解引用读取
}
```

把「此处发生了指针访问」写进表达式，使审计内存访问时可以在文本层面搜索，而不必追踪函数调用的实现。

## 3.15 范围与切片

范围用两个点表示，左闭右开；带等号的形式包含右端点。范围用于循环、分支匹配与切片。

```
for i in 0..10 { }      // 0 到 9
for i in 0..=10 { }     // 0 到 10
var head = xs[0..3]      // 前三个元素
var rest = xs[3..]       // 从第四个到尾
var upto = xs[..2]       // 前两个元素
var all = xs[..]         // 整个表
```

切片的三段形式（只给起点、只给终点、两端都省）使「取头部」「取尾部」「取全部」都有最短写法，不必写出冗长的边界表达式。

下标可以是任意整数类型，宽度不影响语义，超出该类型表示范围的常量下标在编译期即被拒绝。负下标按越界处理——语言不从尾部计数，因此 t[-1] 不会取到末元素，而是与「取到了不存在的位」同样报告。这样下标只有一种读法，读者不必先判断「这个负数在这里是索引还是偏移」。

读越界与除零同属内建检查（见 8.5）：报告源码位置、下标与长度，并沿统一的失败路径处理——在捕获域内可被接住，域外终止进程。切片端点越界则不同，按边界截断，因为「取到尽头」在切片里是常见且明确的意图。

写越界不报错：往末尾之外写会按需扩展容量。值语义下这次写入先复制一份再扩展，因此有别名的一方保持原长度，写入者得到一个更长的表（见 13.5）。

## 3.16 类型判定

类型判定用于询问一个动态类型值的实际类型，配合分支使用，使分支内的取用是安全的。

```
var v: any = 42
if v is i64 {
    var n = as_i64(v)
}
```

判定与转换分开书写，而不是在判定中隐式完成转换。这样每一处的类型假设都是显式的，阅读时不必回推。

### 3.17 集合综合

综合表达式从一个可遍历的值直接构造新表：方括号内写「元素表达式 for 元素 in 来源」，可选地跟一个筛选条件。变换、筛选、收集在一次书写里完成，与遍历语句同构——会写 for 就会写综合。

```
var squares = [x * x for x in 0..10]
var big = [x for x in xs if x > 100]
var names = [u.name for u in users if u.active]
```

综合是普通表达式，可以出现在任何值的位置，也可以嵌套。来源可以是表、映射的键值对、范围，或一切满足遍历约定的类型（见 6.10）。

```
var pairs = [(k, v) for k in cfg]
var flat = [x for row in grid for x in row]   // 嵌套展平
```

综合立即求值，产出实体表；需要在链中懒式逐元素流动的场合，用迭代器适配（见 6.11）。二者分工：综合负责「一口气造出来」，适配负责「边走边算」。需要映射形态的结果时，对综合结果施加映射转换内建，或直接用键值字面量书写。

> 综合与管道是同一件事的两种入口：日常一行能写完的变换用综合；步骤多、需要复用中间名或作用于无限序列时用管道链。语言不比较二者的优劣，按场合选用。

---

## 第 4 章 语句

语句描述控制流与状态的改变。本章逐一说明各类语句，并给出它们在书写上的一致做法。语句之间以自动分号分隔（见 1.10），因此不必为每行补分号，也不应依赖缩进决定结构。

### 4.1 变量声明

变量用 var 声明，常量用 const 声明。常量必须在声明处确定取值，此后不可重新绑定。

```
var count = 0
var name: string = "tie"
const MAX = 1024
```

#### 作用域

变量在所在块内可见，块结束时消失。内层块可以声明与外层同名的变量，此时内层遮蔽外层；编译器对遮蔽给出提示，因为它常是笔误。

函数参数与函数体的局部变量处于同一作用域。闭包可以读取其外层函数的变量，规则见第 5 章。

## 4.2 赋值

赋值把右侧的值写入左侧的目标。复合赋值形式把运算与写入合并。

```
count = 10
count += 1        // 自增
count *= 2        // 倍增
count %%= 5       // 原地取余并写回
```

赋值目标的类型与右侧表达式必须相容。语言不做隐式的窄化转换，需要窄化时显式书写转换。

## 4.3 条件分支

if 接受一个布尔表达式。else if 串接多个条件，else 收尾。条件必须严格是 bool——语言不接受用整数代表真伪，这排除了把赋值误写成比较的一类错误。

```
if score >= 90 {
    grade = "A"
} else if score >= 60 {
    grade = "B"
} else {
    grade = "C"
}
```

本节的 if 是语句：分支描述「做什么」，不产出值。条件取值（if 也可以是一个表达式）见 4.13。

## 4.4 循环

循环有三种形态：条件循环、范围循环、集合循环。三者覆盖了绝大多数遍历需求，因此不需要通用的三段式循环语句。

```
while running {
    step()
}

for i in 0..10 {
    println(to_string(i))
}

for item in items {
    handle(item)
}
```

遍历文本使用码点迭代，这样多字节字符不会被切断：

```
for ch in text.chars() {
    // ch 是一个完整字符
}
```

### 循环标签

嵌套循环中，若需要从内层直接跳出到外层之外，用标签指明目标。标签写在循环语句之前。

```
outer: for a in 0..10 {
    for b in 0..10 {
        if b == 5 { break outer }
        if b == 2 { continue outer }
    }
}
```

无标签的 break 与 continue 作用于最近一层循环。带标签的跳转在编译期校验目标存在，标签未匹配即报错。

## 4.5 多分支选择

switch 按值分派到若干分支。每个分支自动结束，不存在落入下一分支的情况，因此不需要 break。

```
switch code {
    case 1, 2:
        println("小")
    case 3..7:
        println("中")
    default:
        println("其他")
}
```

分支可以匹配枚举并同时取出其携带的数据，此时编译器要求所有分支被覆盖，遗漏即报错。这是把「几种情况之一」的数据处理写完整的主要手段。

```
switch shape {
    case Shape.Circle(r):
        println(to_string(r))
    case Shape.Rect(w, h):
        println(to_string(w * h))
}
```

## 4.6 跳转

return 结束当前函数并带回值；yield 在生成器中产出一个值并继续执行；panic 报告不可恢复的错误并终止当前执行流（见第 8 章）。

```
func pick(xs: table<i64>) -> i64 {
    for x in xs {
        if x > 0 { return x }
    }
    return 0
}

func counter(n: i64) -> i64 {
    for i in 0..n {
        yield i
    }
}
```

goto 在需要把多层嵌套的清理路径收敛到一处时使用。它只允许在同一个函数内跳转，且不允许跳入循环或分支的内部——这两条限制使它不会破坏块作用域。

```
func transfer() -> i64 {
    if open_a() != 0 { unsafe goto #done }
    if open_b() != 0 { unsafe goto #after_b }
    // ...
    #[tag.after_b]
    close_b()
    #[tag.done]
    close_a()
    return 0
}
```

## 4.7 表达式语句

一个表达式后面接语句结束，构成表达式语句。它用于调用有副作用的函数。

```
println("hello")
items -> sort -> print_all
```

没有副作用的表达式语句通常意味着书写疏漏，编译器会对此给出提示。

## 4.8 条件解构

当条件涉及「取值可能失败」的类型时，条件与解构可以合并书写：匹配成功则进入分支并取出内部值，失败则走另一支。

```
if let Some(v) = find(xs, target) {
    println("找到 " + to_string(v))
} else {
    println("未找到")
}
```

这一形式省去了「先判断、再取值」的两步写法，并保证取值只在匹配成立的路径上发生——判断与取值之间不存在状态变化的窗口。

## 4.9 前置条件

guard 在函数开头声明「必须满足什么才能继续」。条件不成立时执行给定的收尾块（通常是返回或报错），成立则继续往下走。

```
func transfer(from: string, amount: i64) -> bool {
    guard amount > 0 else { return false }
    guard from != "" else { return false }
    // 走到这里，两个条件都已成立
    return true
}
```

与 if 的差别在阅读顺序：guard 把前置条件与随后的正常流程分开，使正常路径保持在同一缩进层级，不必层层深入。收尾块必须以跳转语句结束（返回、报错或跳出），否则条件不成立时无法离开当前流程。

## 4.10 作用域收尾

defer 登记一段在离开当前作用域时执行的代码。多段收尾按登记的逆序执行，因此「先申请后释放」的对应关系自然成立。

```
func process(path: string) {
    var h = open(path)
    defer { close(h) }
    var buf = acquire()
    defer { release(buf) }
    // 离开作用域时：先 release(buf)，再 close(h)
    use(h, buf)
}
```

把释放写在申请的下一行，是这一记号的主要用法：申请与释放成对出现在同一屏内，中间无论有多少条提前返回的路径，都不会漏掉释放。

## 4.11 错误聚合块

一段代码中若有多处可能失败的操作，可以把它们放进一个块内统一处理：块内每个可能失败的操作后面写传播记号，块本身可以取到最后一个表达式的值。

```
var total = try {
    var a = read_first() ?
    var b = read_second() ?
    a + b
}
```

## 4.12 跳转与标签

在需要把多层嵌套的清理路径收敛到一处时使用跳转。标签声明某处可以被跳转抵达，跳转语句引用该标签。

```
func retry_read() -> i64 {
    #[tag.retry]
    var r = read()
    if r == 0 { unsafe goto #retry }   // 回到同一块的开头重试
    #[tag.done]
    return finish(r)
}
```

跳转受三条限制：只能在同一函数内跳转；只能跳到同一块或已经执行过的外层块；不得跳过变量初始化进入其作用域。这三条共同保证跳转不会破坏作用域的有效性。跳转属不安全操作，需要显式标注。

## 4.13 分支即表达式

多分支不仅可以作语句，也可以作表达式：每个分支给出一个值，整个分支结构即具有该值。它使「按情形取值」不必先声明变量再逐个赋值。

```
var label = switch n {
    0 -> "零",
    1..9 -> "个位",
    _ -> "多位",
}
```

作表达式时分枝用箭头书写，分支之间以逗号分隔；分支必须穷尽，未覆盖的情形由通配分支承接。作语句时分支用冒号书写，形态见 4.5。

双分支的条件取值有更直接的形式：if 表达式。条件成立时取前一臂的值，否则取 else 臂的值，未成立的臂不会被求值。else 不可以省略——表达式在任何情形下都必须有一个值；允许省略 else 的条件分支是语句，形态见 4.3。条件同样必须是 bool。

```
var fee = if count > 0 { base * count } else { base }
```

两臂取值的类型须一致；同为整数族或浮点族时允许混写，统一到较宽的一方，其余情形要求完全一致。条件多于两个时用 else if 串接，按书写顺序判定，取第一个成立分支的值。if 表达式与 switch 表达式互补：各条件彼此独立时用 if，对同一对象按取值分派时用 switch。

### 4.14 解构

把复合值的各分量一次绑定到多个名字，称为解构。语言在四种位置支持解构，形态一致。

| 位置 | 写法 |
|---|---|
| 声明 | var (a, b) = pair |
| 声明（剩余） | var (a, ...rest) = xs |
| 遍历 | for (k, v) in pairs { } |
| 赋值 | (a, b) <- pair |

剩余形态把未被列举的尾部收进一个表，用于「取头部若干项、其余整体处理」的场合。

## 第 5 章 函数

函数是行为的单位。本章说明函数的定义与调用形态，并重点说明参数——参数是调用方与被调方之间的契约，写好参数规则，函数的行为边界就清楚了一半。

### 5.1 定义与调用

函数定义给出名称、参数与返回类型；返回类型不可省略。函数体是语句块，最后一条语句通常是 return。

```
func add(a: i64, b: i64) -> i64 {
    return a + b
}

var s = add(1, 2)
```

对外可见的函数标注 pub。可见性同时受模块与命名空间约束（见第 9 章）。

## 5.2 参数传递约定

参数的传递约定写在参数名的位置，共四种：按值、可写借用、只读借用、所有权转移。约定是声明的一部分，调用方不需要也无法改变它。

| 形态 | 语义 |
| --- | --- |
| x: T | 按值。被调方得到一份副本，改动不影响调用方。默认约定。 |
| ref x: T | 可写借用。被调方就地读写调用方的对象，改动对调用方可见。 |
| immut x: T | 只读借用。被调方只能读取，不能写入；用于避免大对象的复制开销。 |
| move x: T | 所有权转移。实参的所有权移交被调方，调用方此后不得再使用该变量。 |

```
func by_value(x: i64) -> i64 { return x }
func by_ref(ref t: table<i64>) -> i64 { t <- 1; return len(t) }
func by_read(immut t: table<i64>) -> i64 { return len(t) }
func by_move(move t: table<i64>) -> i64 { return len(t) }
```

### 约定的互斥与组合

可写借用与只读借用互斥：同一个参数不能既允许写又声明只写。所有权转移与两者也互斥：所有权已经交出，无从借用。同一个修饰重复书写视为笔误，编译器报告错误。

> 四种约定是互斥的完备划分——每个参数恰好属于其中一种。这一设计使得阅读函数签名即可知道它对实参做了什么，不需要查看函数体。

### 可写借用的限制

可写借用要求实参是可寻址的对象（变量、字段、表元素），而不是临时值。这样「改动会写回」这一承诺才有着落。字面量与调用结果不能作为可写借用的实参。

### 所有权转移后的使用

实参被移出后，调用方再使用该变量会被检出。这条检查在移动语义检查阶段执行，覆盖直接使用与间接使用。

```
var data: table<i64> = []
data <- 1
var n = by_move(data)
println(to_string(len(data)))   // 错误：变量已移出
```

## 5.3 默认参数

参数可以带默认值。带默认值的参数必须连续排在参数表末尾，因此调用方只需按顺序给出前面的实参。

```
func connect(host: string, port_no: i64 = 8080, timeout: i64 = 30) -> i64 {
    // ...
    return 0
}

connect("localhost")            // port_no 与 timeout 取默认
connect("localhost", 9090)      // timeout 取默认
connect("localhost", 9090, 60)  // 全部给定
```

默认值在编译期确定，因此它必须是常量表达式——不含副作用，也不依赖调用点的作用域。这一限制使得默认值可以安全地在多个调用点之间共享。

可写借用与所有权转移的参数不带默认值：借用需要一个真实对象，所有权转移需要一个真实来源，两者都无法凭空构造。

## 5.4 命名实参

调用时可以按参数名指定实参。命名实参写在位置实参之后，可以乱序。这样做的好处是调用点自解释——不必回看函数定义即可知道每个值对应什么。

```
connect(host: "localhost", timeout: 60)
connect("localhost", timeout: 60)
```

命名实参可以跳过中间带默认值的参数。上例第二个调用中 port_no 未给出，取默认值。这一规则与默认参数「连续居尾」的约定配合，使得「只改其中一项」的调用写起来最短。

调用形式的三条规则：位置实参在前、命名实参在后；同一个参数不得被指定两次；命名不得与已被位置实参占用的参数冲突。违反任一条即报错，并指明具体位置。

### 与方法的一致性

方法调用（见第 6 章）的命名实参与默认参数规则与函数完全一致。接收者不参与命名——它由调用对象给出，也不能被默认值补齐。

## 5.5 变长参数

参数表末尾可声明变长参数，接收零个或多个同类型实参。变长参数不带默认值，也不能是借用。

```
func sum(rest: ...i64) -> i64 {
    var total = 0
    for x in rest {
        total += x
    }
    return total
}

sum()
sum(1, 2, 3)
```

调用时可用展开记号把一个表摊开成实参序列（见 5.14）。

## 5.6 返回值

函数可以返回多个值，返回类型写成元组。调用方解构接收。

```
func divmod(a: i64, b: i64) -> (q: i64, r: i64) {
    return (a / b, a % b)
}

var (q, r) = divmod(7, 3)
```

元组分量的名称是可选的，但推荐书写：它同时作为调用点的解构依据与文档。

## 5.7 一等函数

函数可以作为值传递、返回与存放。函数类型的写法是 fn(参数类型...) -> 返回类型。

```
func apply(g: fn(i64) -> i64, v: i64) -> i64 {
    return g(v)
}

func twice(g: fn(i64) -> i64, x: i64) -> i64 {
    return g(g(x))
}
```

具名函数可直接用作函数值，不需要额外的取地址或包装。

## 5.8 闭包

闭包是携带环境的函数值。它由函数字面量产生，可以读取其定义处的变量。

```
var base = 100
var bump: fn(i64) -> i64 = func(t) -> i64 { return t + base }
println(to_string(bump(1)))   // 101
```

闭包字面量的参数类型在上下文给出函数类型时可以省略，由上下文补齐。没有上下文时报错，而不是猜测一个类型。

## 5.9 短闭包

单参数、单表达式的闭包可以写成短形式：参数名、箭头、表达式。

```
var f: fn(i64) -> i64 = t -> t + 1
var g: fn(i64) -> i64 = n -> n * n + 1
```

短闭包可用在声明位置（标注给出类型）与赋值位置（目标类型在语义分析时已知）。这两种位置都要求一个函数值，因此箭头两侧的解释是确定的。完整形态与短形态可以混用，二者语义相同。

## 5.10 捕获面

闭包能捕获什么，由捕获面白名单规定。这条规则的目的很具体：保证逃逸出定义处的闭包不会持有已经失效的引用。

| 可捕获 | 说明 |
| --- | --- |
| 外围的按值参数 | 标量、结构与枚举捕获值快照；表与映射按引用捕获 |
| 外围的只读借用参数 | 同上；且不可写回 |
| 外围的局部变量 | 同按值参数的划分 |
| 常量 | 常量本身，无副作用 |
| 全局变量 | 始终可见，不经捕获通道 |

可写借用参数不在白名单内，这是本章最需要记住的一条。原因不是书写风格，而是语义：可写借用不是副本，而是指向调用方对象的引用。若闭包捕获了它并逃逸出去，引用所指的对象可能已经不存在。

```
// 错误：闭包捕获可写借用参数
func make(ref t: table<i64>) -> fn() -> i64 {
    return func() -> i64 { return len(t) }
}
```

在不安全上下文中，这条限制放宽——那里由书写者承担引用有效性的责任（见第 11 章）。

### 捕获的语义

捕获分两类。标量、结构与枚举按值捕获：捕获发生在闭包创建的时刻，此后外围变量再改变，闭包看到的仍是创建时的值。表与映射按引用捕获：闭包与外围看到同一份底层数据，闭包内对内容的修改外围可见，反之亦然。语言不为容器做隐式复制——复制容器是显式操作，它的成本必须由书写者看见而不是被语言悄悄支付。

> 表按引用捕获意味着闭包可能形成共享可变状态：这是本规则唯一的例外通道。把携带容器的闭包交给其他执行流之前，必须确信不存在并发写入，或把访问收敛到单一执行流（见第 10 章）。需要彻底隔离时，显式复制一份再捕获。

## 5.11 外部函数

extern 声明外部实现的函数，用于调用操作系统或第三方库。声明给出名称与签名，不含函数体。

```
extern func getenv(name: string) -> i64
```

外部函数可能返回指针或写入调用方的缓冲区，因此调用点通常需要不安全上下文。对系统调用与 C 库的封装应当集中在少数模块内，其余代码使用封装后的安全接口。

## 5.12 等号体与表达式体

函数体有两种简洁形态：体是单个表达式时，可以省略花括号直接写在等号之后；或者写在花括号内而不写返回语句。二者语义相同——都等价于把该表达式的值返回。

```
func twice(x: i64) -> i64 = x * 2
func square(x: i64) -> i64 { x * x }
func log(x: i64) -> void = println(to_string(x))   // 无返回值：值被丢弃
```

返回类型必须显式标注，不因体简化而省略——返回值是函数契约的一部分，不应当需要读者去推导。需要多条语句、或需要提前返回时，仍使用完整的块体。

### 5.13 隐式单参闭包

短闭包还有一种更短的形态：单参数的闭包可以省略参数名，直接使用约定的名字指代该参数。适用的场合是上下文已经给出函数类型、而参数名对可读性无所增益时。

```
var doubled = map(xs, { it * 2 })
var names = map(users, { it.name })
```

参数类型与返回类型由上下文的函数类型推定。上下文不明确时报错，而不是猜测一个类型——这是语言在「简洁」与「可诊断」之间的固定取舍。

### 5.14 实参展开

调用时可以把一个表的元素铺开成多个实参，用于把已有数据转交给变长形参。展开记号在元素前后均可使用，多个展开可以并列。

```
var xs: table<i64> = [1, 2, 3]
var t = sum(xs...)            // 全部铺开
var u = cat(5, 6, xs..., 100)  // 与固定实参混用，展开在中间
var v = sum_all(a..., b...)    // 多个展开依序平铺
```

## 第 6 章 数据结构

tie 在结构设计上采取一条明确的分离原则：结构只描述数据，逻辑写在命名空间函数里，二者通过名称绑定。这条原则使「这是一个什么东西」与「可以对它做什么」成为两件独立的事，各自可以单独演进。

### 6.1 结构的定义

结构由字段组成，字段有类型与可选的默认值。结构是纯数据——不包含方法、不包含虚表、不引入隐式状态。

```
struct Point {
    var x: f64 = 0.0
    var y: f64 = 0.0
    var label: string = ""
}
```

字段的默认值使得构造可以只给出关心的部分，其余取默认。

### 6.2 构造与字段访问

构造按字段顺序给出实参。字段访问用点号。

```
var p = Point(1.0, 2.0)
p.x = 3.0
println(p.label)
```

结构作为参数时默认按值传递；对较大的结构使用只读借用以避免复制，需要修改时使用可写借用。

## 6.3 逻辑与数据分离

结构的行为写在同名命名空间内。命名空间函数的第一个参数是接收者。

```
namespace Point {
    pub func dist(p: Point) -> f64 {
        return sqrt(p.x * p.x + p.y * p.y)
    }

    pub func move_by(ref p: Point, dx: f64, dy: f64) {
        p.x += dx
        p.y += dy
    }
}
```

这一组织方式有一个直接好处：结构定义保持稳定，而逻辑可以增补、分组、按需导入。新增一个操作不需要改动结构所在的文件。

## 6.4 方法调用

点号调用是对命名空间函数的转发：编译器把 obj.method(args) 解释为「与 obj 从属的命名空间下的 method，接收者作为第一个参数」。前提是 「obj 有成员可谈从属」——结构（字段与同名命名空间的方法）、枚举（变体与方法）、命名空间与文件（成员）。「纯数据值不设点」：表与字符串没有成员，它们的操作写成自由函数 `op(xs, …)`（如 `table_pop(t)`），或送进箭头 `xs -> op(…)`（数据作末参）。

```
var p = Point(3.0, 4.0)
println(p.dist() -> to_string)

p.move_by(1.0, 1.0)

table_pop(t)          // 表操作：自由函数
t -> table_pop()      // 表操作：箭头（数据作末参）
```

转发意味着方法的参数规则与普通函数完全一致——默认参数、命名实参、借用与转移约定都可以用在方法上。接收者不参与命名，也不能被默认值补齐：它由调用对象给出。

```
struct Counter { var n: i64 = 0; }

namespace Counter {
    pub func bump(c: Counter, step: i64 = 1) -> i64 {
        return c.n + step
    }
}

c.bump()              // step 取默认
c.bump(step: 4)       // 命名实参
```

接收者的传递约定由命名空间函数声明决定：需要修改调用对象时把接收者声明为可写借用。

自由函数形态与方法形态语义完全相同，可以互相改写。正文示例以方法形态为主——它让「谁在做什么」读起来最直白；自由函数保留两个用途：机器生成代码的稳定形态，以及管道中把函数本身作为值传递的场合。

## 6.5 继承

结构可以继承另一个结构的字段，形成「更具体的类型」。继承只复用字段，不引入方法表——行为仍然由命名空间函数提供。

```
struct Shape {
    var name: string = ""
}

struct Circle extends Shape {
    var r: f64 = 0.0
}
```

子类型可以传给期望父类型的参数，字段按前缀布局，因此转换不需要额外工作。

## 6.6 语义限制

以下限制在编译期检查，它们共同保证结构是纯数据的这一性质。

- 结构体内不得定义函数；行为写在同名命名空间内
- 字段名在同一结构内唯一
- 继承链不得成环
- 字段默认值必须是常量表达式
- 结构作为参数时不得隐式发生窄化转换

> 这些限制不是风格偏好，而是设计后果：结构不携带行为，就没有虚表、没有构造顺序问题、没有隐式析构。调试时看到一个结构值，就看到了它的全部状态。

## 6.7 与元组的关系

元组适合表达「几个值临时组合」；结构适合表达有名字、会长期存在、有自己行为的概念。判据是命名与复用：需要按名字取用、需要在多处传递，就用结构。

```
// 临时组合：用元组
var (lo, hi) = min_max(xs)

// 长期概念：用结构
struct Interval {
    var lo: i64 = 0;
    var hi: i64 = 0;
}
```

## 6.8 运算约定

结构可以定义运算符的含义。做法是在同名命名空间里提供约定名称的函数，编译器在遇到相应运算时调用它。这样自定义类型可以像内建类型一样书写，而不必到处写方法调用。

| 约定名 | 触发的运算 |
| --- | --- |
| add / sub / mul / div | 相应的二元算术 |
| neg | 一元取负 |
| eq / cmp | 相等比较与大小比较 |
| index / index_set | 下标读取与下标赋值 |
| to_string | 转为文本 |

```
namespace Vec2 {
    pub func add(a: Vec2, b: Vec2) -> Vec2 {
        return Vec2(a.x + b.x, a.y + b.y)
    }
    pub func eq(a: Vec2, b: Vec2) -> bool {
        return a.x == b.x && a.y == b.y
    }
}

var v = p + q          // 调用 Vec2::add
if p == q { }          // 调用 Vec2::eq
```

约定名称是一份固定清单，不接受任意名称。约定函数是普通函数，可以单独调用——运算符只是它的另一种写法，不存在只能通过运算符访问的行为。

## 6.9 计算属性

对需要按需计算、不宜存储的值，可以定义一对同名函数作为读写入口：读函数除接收者外无参数，写函数多一个值参数。访问时用普通的字段语法。

```
namespace Circle {
    pub func area(c: Circle) -> f64 { return 3.14159 * c.r * c.r }
    pub func label(c: Circle) -> string { return "圆 " + to_string(c.r) }
    pub func label_set(ref c: Circle, v: string) { c.name = v }
}

var a = c.area         // 读：调用计算
```

读入口与普通字段访问在调用点写法一致，因此这个值究竟是存下来的还是算出来的属于实现细节，改动它不影响使用方。写入口需要接收者可写借用，因为写入必然修改对象。

## 6.10 遍历约定

要让自定义类型可用于遍历语句，在命名空间里提供两个约定函数：一个判断是否还有元素，一个取出当前元素并前进。有了它们，该类型就能出现在遍历语句中。

```
namespace Ring {
    pub func has_next(r: Ring) -> bool { return r.i < r.n }
    pub func next(ref r: Ring) -> i64 { r.i += 1; return r.buf[r.i - 1] }
}

for v in ring { println(to_string(v)) }
```

约定遍历的行为与内建表一致：遍历中的修改规则、元素顺序、取尽后的判定都遵循同一套语义，因此

使用方不必区分遍历的是表还是自定义类型。

此外，形如 `fn() -> Option<T>` 的值（流，见 6.11）同样可以直接遍历——每次调用取下一个元素，取尽以 None 表明。

## 6.11 迭代器适配

遍历约定之上还有一层适配函数，用于把「逐个取元素」组合成常用的变换。适配函数是自由函数，作用于一切满足遍历约定的类型——内建表与自定义类型同等对待。

| 函数 | 语义 |
| --- | --- |
| iter_map(f, xs) | 逐元素变换，产出 f(元素)；返回新表。 |
| iter_filter(p, xs) | 保留判定为真的元素；返回新表。 |
| iter_take(n, xs) | 取前 n 个元素；返回新表。 |
| iter_zip(a, b) | 两路并读，产出 (a_i, b_i) 元组，短者取尽即止。 |
| iter_chain(a, b) | 先后串接两路元素。 |

```
var top = scores -> iter_map(func(v: i64) -> i64 { return v * 2 }) ->
iter_filter(func(v: i64) -> bool { return v > 0 })
for (name, score) in iter_zip(names, scores) {
    println(name + ": " + to_string(score))
}
```

惰性流

上表的适配函数逐元素物化，适合表内数据。需要在大序列上短路求值时（例如「取前十个」而源还有余量），用「流」：流是一个每次调用返回下一个元素的函数——有素即 `Option.Some(元素)`，取尽恒 `Option.None`。表经 `stream_from` 成流；适配链逐元素传递，`stream_take` 达到数量后不再向上游拉取；遍历语句直接消费流（有素绑定元素执行循环体、取尽退出）。适配函数：`stream_map`、`stream_filter`、`stream_take`（保持流形态）；`stream_collect`（成表）、`stream_each`（逐元素应用）收束。

```
var top2 = scores -> stream_from() -> stream_take(10) -> stream_collect()
for v in stream_from(scores) -> stream_take(3) {
    println(to_string(v))
}
```

适配器是惰性的：组合若干适配器不产生中间表，元素在被遍历到的那一刻才计算并流动。这条约定让「变换链」的成本与手写循环相当，也使适配器可以安全作用于无限序列（如计数源）。需要把结果固化为表时，用 table_from(xs) 收拢。

# 第 7 章　接口

接口描述一组能力：满足这组签名要求的类型，就可以被当作这个接口使用。接口是泛型约束与多态的基础，也是模块之间约定边界的主要手段。

## 7.1 声明

接口用 port 声明，其中列出方法签名。签名只写参数与返回类型，不含实现。

```
port Drawable {
    pub func draw(self, ctx: i64) -> string
    pub func bounds(self) -> i64
}
```

self 是接收者占位，实现时由具体类型替换。

## 7.2 实现

类型通过 impl 块声明它实现某个接口，并在其中给出各方法。接口要求的每个方法都必须实现，遗漏在编译期报告。

```
struct Button {
    var label: string = ""
    var w: i64 = 0
    var h: i64 = 0
}

impl Drawable for Button {
    pub func draw(self, ctx: i64) -> string {
        return "Button(" + self.label + ")"
    }
    pub func bounds(self) -> i64 {
        return self.w * self.h
    }
}
```

实现块中的 self 指向实现类型本身，因此实现体可以直接访问该类型的字段。

## 7.3 作为约束

泛型参数可以标注接口约束。约束在编译期检查：传入的类型必须实现了该接口，否则报错。

```
func render_all<T: Drawable>(items: table<T>, ctx: i64) {
    for it in items {
        it.draw(ctx)
    }
}
```

带约束的泛型在编译期为每个具体类型生成一份实现。因此调用没有间接跳转，也能被内联——接口在这里不带来运行期开销。

## 7.4 作为对象

接口也可以作为运行期的对象类型使用：把不同具体类型的值放入同一个接口类型的变量或表中，调用时按实际类型分派。

```
var items: table<Drawable> = []
items <- button
items <- label
for it in items {
    it.draw(0)
}
```

这一形态在需要异质集合时使用，例如界面中混合放置不同控件。开销与任何动态分派方案相同，因此在不必要时优先使用泛型约束的形态。

> 两种形态共用同一份接口声明。选择哪一种取决于调用点是否需要异质集合：类型在编译期全部已知，用约束；需要把不同类型放进同一个容器，用对象。

## 7.5 接口的组合

接口可以要求另一个接口，从而在能力之上再叠能力。实现时必须同时满足全部要求。

```
port Focusable {
    pub func focus(self) -> bool
}

port Control {
    pub func draw(self, ctx: i64) -> string
    pub func focus(self) -> bool
}
```

组合让约束可以按能力分片：小接口易于实现也易于复用，大接口由小接口拼成，实现方仍只需提供具体方法。

## 7.6 两种接口

接口有两种形态，区别在于「要求是否必须被显式声明」。

| 形态 | 要求 |
| --- | --- |
| 名义接口 | 类型必须显式声明它实现了该接口。声明与实现分开书写，实现块中逐方法给出定义。 |
| 结构接口 | 只要类型具备接口要求的方法，即视为满足。无需显式声明。 |

```
// 名义接口：必须 impl
port Drawable { pub func draw(self, ctx: i64) -> string }
impl Drawable for Button { pub func draw(self, ctx: i64) -> string { return "B"
} }

// 结构接口：具备方法即可
interface Named { pub func name(self) -> string }
// 任何有 name(self) -> string 的类型都满足 Named
```

选择依据是意图。名义接口用于表达「这是一份契约，实现方必须知情」；结构接口用于表达「这个算法只要求这些能力」，此时关心的是能力而非声明。二者的约束检查都在编译期完成，也都能作为泛型约束使用。

# 第 8 章　错误处理

tie 把「可预期的失败」与「不可恢复的故障」分开处理。前者是返回值的一部分，调用方必须面对；后者终止执行并给出位置信息。这样区分的原因是二者的处理方式根本不同：前者需要分支决策，后者需要诊断信息。

## 8.1　可预期的失败

可能失败的函数把失败编码进返回类型。语言提供两个通用型别用于此目的：一个表示「有值或无值」，一个表示「成功值或错误值」。

```
func find(xs: table<i64>, target: i64) -> Option<i64> {
    for x in xs {
        if x == target { return Some(x) }
    }
    return None
}

func parse_port(s: string) -> Result<i64, string> {
    if s.len() == 0 { return Err("空输入") }
    return Ok(to_i64(s))
}
```

## 8.2　传播

当函数无法就地处理失败、需要把它交给调用方时，用问号后缀传播：成功则取出值继续，失败则立即返回。

```
func config_port(cfg: map<i64>) -> Result<i64, string> {
    var raw = cfg["port"]?
    var port_no = parse_port(raw)?
    return Ok(port_no)
}
```

传播记号把「错误向上传递」这条最常见路径写成一个字符，从而让函数体里的正常路径保持直线——不必为每一层调用写一段相同的分支。

## 8.3　分支处理

需要针对具体情况分别处理时，用多分支匹配展开结果类型。编译器要求覆盖全部情形。

```
switch parse_port(input) {
    case Ok(v):
        println("端口 " + to_string(v))
    case Err(msg):
        println("失败：" + msg)
}
```

对于「缺失则取默认」这一常见需求，用安全默认值记号直接表达，不必展开分支。

```
var port_no = cfg["port"] ?: 8080
```

## 8.4 标准组合子

标准库提供一组转换与组合函数，用于在结果类型之间衔接，减少手写分支。

```
var doubled = parse_port(s) -> map_result(v -> v * 2)
var len_ok = parse_port(s) -> and_then(validate_range)
```

## 8.5 不可恢复的故障

当故障意味着「程序状态已经不可信」时，用 panic 终止。它携带一条消息，用于定位问题。

```
if len(buf) == 0 {
    panic("缓冲区不应为空")
}
```

在明确划定边界的场合可以捕获 panic 并转为普通错误，例如服务的请求处理循环：单个请求的故障不应终止整个进程。捕获点应当少而集中，且必须能说明为什么在此处恢复是安全的。

### 内建检查

表与数组的下标越界（含负下标）、除零，以及带检查运算的溢出，都会触发内建检查。检查报告源码位置与相关的量——下标越界给出下标与长度，除零给出发生处，使故障不必靠复现去猜。

这些检查与 panic 走同一条失败路径：在捕获域内被接住并转为普通错误，域外终止进程。它们不可关闭，因为其成本相对故障后果可以忽略。

## 8.6 错误信息

错误值应当说明发生了什么、在哪一步发生。把上下文带在错误里，比在每一层重新描述问题更有用。

```
return Err("读取配置失败：字段 " + name + " 缺失")
```

> 错误类型的选取遵循一条经验：能枚举的用枚举，不能枚举的用文本。当调用方需要按类别分支时用枚举；当调用方只是向上传递或打印时用文本。

## 8.7 错误链

错误在层层传播中会远离它的根因。语言为此提供包装机制：错误值可以携带一个下层错误，形成一条链。链的最外层说明「这一步做什么失败了」，逐层向下是「为什么失败」，链底是根因。

| 函数 | 语义 |
| --- | --- |
| r.wrap_err(msg) | 作用于 Result：Ok 原样通过，Err 以新消息包装并挂入下层错误。 |
| e.cause() | 取下层错误；无下层时返回 None。 |
| e.root_cause() | 沿链取到最底层根因。 |

```
func load_config() -> Result<Conf, string> {
    var raw = read_file(path) -> wrap_err("读取配置失败")?
    return parse_conf(raw) -> wrap_err("配置解析失败")?
}

switch load_config() {
    case Err(e):
        println(e)                  // 给用户看：最外层消息
        println(e.root_cause())     // 给排障看：最初根因
}
```

> 包装的纪律是逐层补充语境、不重复内容：每一层只说这一步发生了什么，不抄写下层已有的消息。展示给用户的取最外层，日志与排障的取根因，中间层只在逐层排查时才展开。

---

# 第 9 章 模块与导入

程序由多个文件组成，文件之间通过导入建立依赖。本章说明导入的语法、可见性规则与命名空间的角色。

## 9.1 导入

导入把另一个文件的内容引入当前文件，并可选地给它一个别名。别名用于限定访问，使来源在调用点清晰可见。

```
import "./geometry.tie" as geo
import "./format.tie"

var d = geo.distance(a, b)
```

不写别名时，被导入文件的公开名称直接进入当前作用域，适合导入内容很少的文件。来源较多时使用别名，避免名称冲突。路径有两种写法：相对当前文件以 ./ 起头（./geometry.tie），标准库模块以库根起头（/std/string.tie）；两者都写到文件名并带扩展名。

## 9.2 命名空间

命名空间把一组函数组织在同一个名称之下。命名空间可以跨越多个文件：若干文件声明同一个命名空间时，它们的内容合并为一份。

```
namespace math {
    pub func gcd(a: i64, b: i64) -> i64 {
        if b == 0 { return a }
        return gcd(b, a % b)
    }
}
```

命名空间可以嵌套，形成层次化的名称。嵌套的层级同时表达归属关系：内层命名空间只在外层可见的范围内被引用。

> 跨文件的分片需要由主文件显式导入分片，命名空间才会合并。只在别处导入分片而主文件未导入它，会得到「名称未定义」——这是导入关系的正常结果，不是缺陷。

## 9.3 可见性

声明默认是私有的，只有标注 pub 才对外可见。可见性在导入边界上检查：被导入文件的私有名称不出现在导入方的作用域中。

```
func helper() { }        // 仅本文件可见
pub func api() { }       // 对外可见
```

私有是本语言在处理跨文件访问时的默认选择。理由很实际：默认私有使得一个文件内部的调整不会意外影响其他文件，公开面需要显式付出决策成本。

## 9.4 标准库

标准库以模块为单位提供。常用模块与其职责如下。

| 模块 | 职责 |
| --- | --- |
| string | 文本查找、切分、替换、大小写 |
| utf | 码点与字节的转换、字符分类 |
| collection | 表与映射的常用操作 |
| sort | 排序与查找 |
| math | 数值函数与统计 |
| regex | 正则匹配与替换 |
| fs | 文件与目录 |
| path | 路径拼接与分解 |
| json | 数据交换格式的读写 |
| time | 时间与日期 |
| random | 随机数 |
| process | 进程与环境 |
| net | 网络通信 |
| crypto | 散列与加密 |

### 默认可见的内建

有一批内建不经过导入即可使用：全部内建函数族（len、to_string、str_ 系、sort 系、iter_ 系等，见各章定义），以及结果类型的四个构造名。构造名以裸名提供，书写错误分支时不必每次写全限定名。

```
func parse(s: string) -> Result<i64, string> {
    if s.len() == 0 { return Err("空输入") }
    return Ok(to_i64(s))
}

func first(xs: table<i64>) -> Option<i64> {
    if len(xs) == 0 { return None }
    return Some(xs[0])
}
```

Ok、Err、Some、None 是裸名而非关键字：它们默认可见，也可以被同名声明遮蔽。带限定的全名 Result.Ok 与裸名等价，跨模块书写或需要消除歧义时用全名。

## 9.5 依赖与版本

包管理器按版本解析依赖，并把解析结果与校验信息记录在项目文件中。依赖的引入应当在项目级完成，而不是在源文件中按路径直接引用。

```
// 项目文件（tie.pkg）中记录依赖与版本约束
name: my-project
require:
    json    ^2.1
    net     ~1.4
```

版本约束使用语义化的范围记号，使得依赖升级的边界在文件里可见。

## 9.6 选择性导入与重导出

导入可以只引入需要的名字，也可以给整个文件起别名，还可以把导入的名字再导出给使用本文件的人。

```
import "/std/string.tie"                // 整个模块，用模块名限定访问
import "/std/string.tie" as str         // 起别名
import "/std/string.tie".{ join, repeat } // 只引入列出的名字
pub import "/std/json.tie"               // 重导出：使用本文件者也能看到它
```

选择性导入用于收窄依赖面：文件开头一行即可看出这份代码实际用到了外部模块的哪些能力。重导出用于把若干模块聚合成一个对外入口，使用方只需导入一处。

## 9.7 引入命名空间

using 把某个命名空间的名字直接引入当前作用域，此后引用其中的名字不必再加前缀。

```
using args
using time
```

它适用于使用频繁、且不会与其他名字冲突的命名空间。作为对照，导入别名不改变名字的可见性——别名只是缩短前缀，读者仍能从调用点看出名字来自哪个模块。是否使用 using 是一个可读性判断：让来源显而易见，比少写几个字符更重要。

## 9.8 数据格式

语言自带一种数据表达格式，用于配置文件、接口描述与数据交换。它的语法就是 tie 的表与字面量语法——不需要学习第二套写法，也不需要为数据单独维护一套解析器。

格式有两种载体：可读的文本形式与等价的二进制形式。二者表达同一份数据，前者便于阅读与版本对比，后者便于机器读写与传输。同一份定义可以在两种形式间转换，语义不变。

```
// 可读形式：与 tie 的表字面量同形
api = [
    [ "id": "tsci.linalg", "version": "0.1.0" ],
    [ "funcs": [
        [ "name": "mat_mul", "in": 2, "out": 1 ],
    ],
    ],
]

// 读取：得到一个普通映射，之后按映射规则访问
var id = api[0]["id"]
```

这种格式的定位是生态内的通用数据面：接口定义、构建配置、测试语料都用它表达，因而工具链只需理解一种格式。明文与二进制两种载体的完整规范——td 语法、zd 序列化、tink 帧协议与 tsha1 哈希家族——见第 17 章。

## 9.9 依赖声明

项目依赖写在项目文件里，含名称与版本约束。版本约束用语义化的范围记号表达，升级边界因此在文件里可见。源文件本身按模块名导入，不携带版本信息——版本解析是项目级的事，不该散落在各处。

```
// 项目文件（示意）
name: my-project
require:
    json    ^2.1
    net     ~1.4
    crypto  >=3.0
```

# 第 10 章 并发

并发在 tie 里由两类构造承担：actor 提供隔离的状态单元，任务图提供带依赖的并行调度。两者的共同前提是「默认不共享可变状态」——共享是通过显式通道或消息传递发生的，不是通过共享内存偶然发生的。

## 10.1 并发的基本立场

数据竞争通常来自两处：两个执行流访问同一块可写内存；或者一处持有指向另一处栈帧的引用。语言在设计上分别封堵：

- 共享可变状态通过消息传递表达，通道有明确的所有权归属
- 可写借用不能被闭包捕获，因此不会随函数值逃逸到别的执行流
- 跨线程传递的值按值复制，不建立隐式别名

## 10.2 状态单元

actor 是并发的基本单位：它拥有自己的状态，只通过消息与外界交互。对它的状态的访问全部发生在它自己的执行流里，因此不需要锁。

```
actor Counter {
    var n: i64 = 0

    func inc(step: i64) {
        n += step
    }

    func value() -> i64 {
        return n
    }
}

var c = Counter()
c.inc(5)
println(to_string(c.value()))
```

调用 actor 方法即发送消息。消息在队列中排队，按到达顺序处理，因此状态变化的顺序是可预期的。

## 10.3 消息与返回

消息可以携带数据。需要结果时，调用等待回复并把结果作为表达式的值。带返回值的调用与普通调用写法相同，等待是隐式的。

```
var total = c.value()
```

发送方与接收方之间传递的数据按值语义复制或转移，因此不存在双方同时持有同一块可写内存的情形。

## 10.4 任务与依赖

当工作之间存在依赖关系时，用任务图描述：节点是工作单元，边是依赖。调度器按依赖顺序推进，互不依赖的节点并行执行。

```
var g = graph()
var a = g.add(fetch_a)
var b = g.add(fetch_b)
var c = g.add(merge, a, b)      // c 依赖 a 与 b
g.run()
```

任务图中传递的值遵循与 actor 相同的规则：跨节点传递即复制或转移。这使并行部分与串行部分可以自由混合，而不必为边界处的数据归属单独设计。

> 并发构造的选择依据是「状态是否需要长期存在」。需要长期存在并被多方交互的状态放在 actor 里；一次性的并行计算用任务图。

## 10.5 与闭包捕获的关系

第 5 章的捕获面白名单在这里体现为一条直接的安全保证：由于可写借用不可被闭包捕获，任何函数值都不持有对他人栈帧的可写引用。把函数值交给别的执行流时，不可能带入一个可写别名。

这条规则解释了为什么跨线程传递闭包大体是安全的：闭包携带的标量、结构与枚举都是创建时刻的值快照，每个执行流拿到各自独立的副本。唯一的例外通道是容器捕获（见 5.10）——表与映射按引用捕获，多个执行流因此共享同一份底层数据。把携带容器的闭包交给其他执行流时，并发写入由书写者负责排除；需要共享状态时，把状态收敛到状态单元（见 10.2），让访问经由消息串行化，而不是让多个执行流直接触碰同一个表。

## 10.6 建立与调用

建立一个执行体用 run。它取其构造，返回一个句柄；句柄可以复制，多个副本指向同一个执行体，就像同一个地址的多个引用。

```
actor Counter {
    var n: i64 = 0

    pub func inc(step: i64) -> i64 {
        n += step
        return n
    }

    pub async func reset() {
        n = 0
    }
}

var c = run Counter()       // 建立，取得句柄
var v = c.inc(5)            // 同步调用：等结果
c.reset()                   // 异步投递：不等
```

方法默认是同步的：调用方等待被调方处理完并取回结果。标注为异步的方法则是投递即返回——调用方不等，被调方按到达顺序处理。何时需要等结果、何时只需告知，由方法签名直接表达，不由调用点决定。

### 结果的延后取得

异步方法也可以带回结果，此时调用得到一个代表将来会有值的凭据，需要取值时再等待。等待点上若被调方出错，错误在等待处抛出——错误与结果走同一条路径，不必单独设计上报通道。

```
var f = worker.compute(input)   // 异步：立即返回凭据
// ... 其间可做别的事
var r = await f                 // 需要时取值，可能在此抛出被调方的错误
```

不需要结果时直接丢弃该凭据即可，行为等同于投递。这使得要不要结果不影响书写形态，只影响是否等待。

## 10.7 重入

执行体默认按消息逐条处理，一条处理完才取下一条。当某条处理需要等待另一个执行体的回信时，这段等待会挡住队列。若这种相互等待成环，双方就会互锁。

把这类方法标注为重入，表示它在等待期间允许队列继续处理其他消息，从而打破环路。代价是等待期间自身状态可能被改变，因此标注重入需要确认方法内部对状态变化的假设仍成立。

```
actor Router {
    pub reentrant func route(id: i64) -> i64 {
        return downstream.lookup(id)   // 等待期间可继续处理其他消息
    }
}
```

## 10.8 字段初值

执行体的状态字段在声明处给出初值，且初值必须是字面量——数字、布尔、字符、字符串或空表。语言不提供构造方法。

```
actor Cache {
    var hits: i64 = 0
    var name: string = ""
    var items: table<i64> = []
}
```

限制初值为字面量，是为了让状态在建立那一刻就是确定的：外部读不到半构造状态，也不需要为构造顺序设计规则。需要更复杂初始化的情形，用一个显式的初始化消息完成，那次调用与普通消息走同一路径。

## 10.9 数据流图

需要表达若干工作单元与其依赖关系时，用图。图值与它描述的拓扑同构：花括号界定节点，箭头表示前后依赖，并列表示并行分叉，波浪号前缀表示回边。

```
// 图字面量：并行两路，之后汇合，并带一条回边
var flow = { A } - { B } -> { C -> ~ A }

// 表形态：节点与边逐条登记
var g = graph()
var a = g.add(fetch)
var b = g.add(parse, a)
var c = g.add(store, b)
var d = g.add(report, b)
g.run()                       // 按依赖顺序推进；无依赖者并行
```

图本身是值：可以组合、可以传递、可以作为函数值求值。组合产生新图而不修改原图，因此一张图可以在多处安全复用。把值送入图即触发执行，与管道写法一致。

### 检查与查询

图提供一组查询操作，用于在构造后检查结构：节点与边的枚举、连通性、可达性、环路检测、拓扑序、最短路径、关键路径、最大流与最小割。三态标记可以随图传播，用于在图上表达「是、未知、否」

的判定。这些是图值的一等能力，不必自行实现。

# 第 11 章 不安全机制

语言的安全保证建立在「不出现未定义行为」之上。绝大多数代码不需要违反这条前提，因此默认落在安全区内；少数需要直接操作内存、调用外部接口或放宽检查的场合，语言提供一处显式标记：不安全区。

## 11.1 安全区的边界

安全区内不允许出现以下构造——它们在编译期被拒绝，而不是在运行期碰运气。

- 对指针的解引用与写入
- 原始缓冲的分配与释放
- 内存块的批量拷贝与填充
- 易失访存与内联汇编
- 调用标记为不安全的函数
- 闭包捕获可写借用参数
- 非结构化跳转

注意两处不在其列。其一，「指针类型」本身不在其列：声明、比较、传递一个指针值不会引发未定义行为。其二，「产出指针值」的操作也不在其列：取地址、指针偏移、指针与整数之间的转换都是纯值运算，不触碰内存，结果即便无效也只在被解引用时才成为问题，而解引用另有门禁。安全与不安全的这条线，画在「是否要求书写者先满足某个安全前提」上（见 11.8）。

这些限制覆盖了内存不安全的主要来源：悬垂引用、越界访问、类型混淆、数据竞争。

## 11.2 不安全区

不安全区有两种形态：标记为不安全的函数，以及函数体内的不安全块。二者都把「此处由书写者承担内存正确性的责任」写在代码里。

```
unsafe {
    var buf = alloc(16)
    deref_write(buf, as_u8(1))
    var v = as_i64(deref(buf))
    free(buf)
}
```

不安全函数在签名上标注 unsafe。安全代码调用不安全函数必须自己处在不安全区内，这一要求由编译器强制：调用链上每一处跨越都必须显式。

## 11.3 指针操作

指针类型在类型名上体现为 ptr 与元素类型。指针的「分配、读写与释放」在不安全区内进行；而「产出指针值」的一应操作——取地址、偏移、与整数互转——不触碰内存，在安全代码中即可使用（见 11.8）。指针值的声明、比较与传递同理。

```
var p = alloc(64)              // 分配：须不安全区
var q = ptr_add(p, 8)          // 偏移到第 8 字节：安全（纯地址计算）
deref_write(q, as_i64(42))     // 写入：须不安全区
var n = as_i64(deref(q))       // 读出：须不安全区
var a = ptr_to_int(p)          // 转为整数：安全（纯转换）
free(p)                        // 释放：须不安全区
```

指针与整数的双向转换使外部接口的调用可以表达：把地址作为整数传给不接受指针类型的接口，或把返回的整数重新解释为指针。

## 11.4 放宽的检查

不安全区内的部分检查放宽，由书写者承担后果。放宽的范围是明确的、有限的：指针操作相关的检查与借用逃逸的检查。其余检查——类型相容、参数个数、分支完备性——在任何地方都不会关闭，因为它们与内存布局无关。

闭包捕获可写借用的限制在不安全区内解除，用于需要把借用显式传递到回调的场合。此时借用的有效期由书写者保证。

## 11.5 审计友好

不安全机制的形态经过取舍：它不采用「关闭全文件检查」的开关，也不采用隐式的宽松规则，而是要求把每个不安全操作写在标记区内。这样做的目的是让审计可以按文本搜索完成。

- 搜索 unsafe 即可定位全部需要人工复核的位置
- 不安全区的边界在语法上可见，不依赖约定
- 解引用记号使指针访问在语句层面可辨认

> 封装外部接口时，把不安全操作收敛到少数模块，对外只暴露签名安全、行为可预期的接口。这样不安全区的总量可控，复用这些封装的使用方无需关心内部细节。

## 11.6 能力域与凭据

不安全操作按其触犯的安全前提分组，每组称为一个能力域。要执行某个域内的操作，必须持有该域的凭据。凭据是一类特殊的值：它可以转移，但不能复制——因此同一时刻只有一处持有它，不存在权限被悄悄扩散的情形。

| 能力域 | 覆盖的不安全操作 |
| --- | --- |
| mem | 解引用、指针算术、原始分配与释放、取地址。 |
| ext | 调用语言无法核对的外部代码——自行声明的外部函数、交给外部的函数指针、动态装载的库与按地址调用的函数。系统调用与常用库已由语言封装为安全接口，不需凭据。 |
| share | 跨执行流共享可变数据，越过「默认不共享可变状态」的前提。 |
| raw | 内联汇编、易失访存、非结构化跳转——直接绕过编译器对代码的假设。 |
| lock | 对容器的访问去掉运行时锁——越过「并发访问由运行时看护」的前提，正确性由书写者担责。 |

要执行某个域内的操作，编译器检查两件事：所在位置处于不安全上下文（见 11.2），且当前作用域持

有该操作所属域的凭据。两件都满足才放行；缺凭据时诊断会指出该操作属于哪个域、当前持有哪些域，并列出可用的持证写法。持错域的凭据不算数——持 mem 域凭据去做 ext 域的操作同样被拒。

凭据不随作用域之外的边界传递：它不进入函数体内部（闭包体是独立函数，其内不继承外层的凭据），也不从引入方传给被引入的模块（每个文件按自己的头部声明持证）。

取得凭据的方式是显式请求。请求写成一次调用，返回凭据值；此后在一个显式的块内持证操作，块结束时凭据失效。

```
var g = unsafe.get(mem)            // 请求凭据
unsafe use g {                     // 持证块：块内可做该域的操作
    var p = alloc(64)
    \\p = as_u8(1)
}

unsafe with(share) {               // 作用域临时凭据：退块自动回收
    shared_counter += 1
}

unsafe with(lock) {                // 该块内对容器的访问不再取运行时锁
    buf[i] = v
}
```

凭据可以派生：把域名再写一次，得到一份新的凭据，原凭据随之交出（此后不再可用）。也可以绑定到特定对象，表示这份许可只对该对象有效。

```
var sub = g.delegate(mem)          // 派生新凭据（同域）；原凭据交出
var bound = unsafe.get(mem -> buf)  // 绑定到具体对象
unsafe.revoke(g)                   // 撤销：此后凭据不可用
```

撤销作用于凭据本身。撤销一份凭据后，由它（传递）派生出的凭据一并失效，再使用它们会被拒绝；撤销不因凭据已被派生交出而失去对象——一份凭据即便已经通过 delegate 交出去，仍可撤销它，从而收回其下游的全部许可。重复撤销同一凭据、或撤销根本不是凭据的值，都会被拒绝。

### 按模块授权

系统桥接与外部接口封装这类模块通篇都是不安全操作，逐个函数标注会淹没代码。此时在文件头部一次性授权该文件可用的能力域，或在函数与执行体上标注，表示该单元整体持证。

```
type tie<class, unsafe>            // 整个文件处于不安全上下文
type tie<class, unsafe:lock>       // 并授权 lock 域：本文件各函数隐式持该证
type tie<logic, unsafe:ext, unsafe:mem>   // 一次授权多个域（逗号分隔）
#[unsafe.share]                    // 函数级：该函数隐式持证（域写在标注里）
pub func aggregate() -> i64 { ... }
```

持证共有四种写法，粒度由粗到细：文件头的角色声明（整个文件的各函数隐式持证）、函数上的标注（该函数隐式持证）、unsafe use g 持证块（凭据作为值传入，退块回收）、unsafe with(域) 临时凭据块（声明域即持证，退块回收）。前两种适合整块代码同域的场合，后两种适合局部。

文件级授权只作用于这个文件：它被别的文件引入时，授权不会随之扩散过去；反过来，被引入的模块也不继承引入方的授权，要什么域得自己声明。

按模块授权是粗粒度的，因此它的适用范围是「整块代码都属同一域」的场合。审计时这类文件仍然可见——头部或标注本身就是标记，搜索能力域名称即可定位全部授权点。☒

## 11.7 内联汇编

需要直接书写机器指令时用内联汇编。操作数以「方向加约束」的形式声明：哪些是输入、哪些是输出、哪些既读又写，以及它们要占用寄存器还是内存。

```
unsafe {
    asm!('add {dst}, {src}', dst: 输出(寄存器) = v, src: 输入(寄存器) = n)
}

// 操作数方向：输出 / 输入 / 既读又写
// 约束位置：寄存器 / 内存
```

这份声明让编译器知道指令读了什么、写了什么，因而能在汇编块周围正确安排寄存器与内存的可见性。把方向和约束写清楚，是内联汇编可被优化而不出错的前提。

## 11.8 安全边界的重分类

安全与不平等的边界不是固定不变的，它随实现能提供的保证而移动。以下几类操作随着实现成熟，从需要持证变为安全代码即可使用；下列重分类均已生效。

- 原子类型的全部操作（原子读写与比较交换）：此前需 mem 域，现由类型系统保证其正确使用。
- 切片类型与其视图、长度与下标访问：此前需 mem 域，现由边界检查保证——越界是可捕获的失败，不是未定义行为。
- 指针类型的声明、比较、传参与返回：此前需 mem 域，现只需不解引用即安全。
- 产出指针值的一应操作——取地址、取字段地址、指针偏移、指针与整数互转：此前需 mem 域，现为纯值运算，不触碰内存。
- 明确声明内存布局的结构：布局由声明保证，不需持证。

这组重分类的共同点是把边界从「类型」移到「操作」：说得出「一个指针」不等于能对它做危险的事，所以类型本身不再需要持证，而危险操作照旧需要。

顺着这条线再推一步，边界可以收得更准：「一个操作需要持证，当且仅当它要求书写者先满足某个安全前提」——这个前提可以是「指针有效」，也可以是「内存已初始化、未重复释放、长度不越界」。产出指针值的一应操作没有这样的前提，因而全部归入安全区；而解引用、分配与释放、块拷贝、易失访存各有前提，一律维持持证。

仍维持需要持证的是：对指针的解引用与写入、原始缓冲的分配与释放、内存块的批量拷贝与填充、易失访存、内联汇编、非结构化跳转、调用语言无法核对的外部代码、跨执行流共享可变数据，以及对容器访问去掉运行时锁。

这条边界线的划法是：凡实现能够自行保证其安全的，就归入安全区；凡安全性取决于书写者判断的，就维持持证。这条线只随实现能力移动，不随使用便利移动。☒

# 第 12 章 宏与元编程

宏让代码可以生成代码。tie 的宏在编译期执行，输入与输出都是代码片段——代码在这里被当作普通的值来传递，因此不需要为模板另造一套语言。

## 12.1 代码片段

code 是编译期类型，它的值是一段语法结构，而不是运行期的数据。code 值只能出现在宏相关的上下文里，不能存进变量跨阶段使用。

```
macro twice(x: code) -> code {
    return `(($x) + ($x))
}
```

## 12.2 宏的定义

宏用 macro 声明。宏在编译期被调用，其返回值替换调用点。由于展开发生在编译期，宏不带来运行期开销。

```
macro unless(cond: code, body: code) -> code {
    return `{ if !($cond) { $body } }
}

unless(done, cleanup())
```

## 12.3 引用与插值

引用记号把一段代码包成 code 值而不立即求值；插值记号把 code 值嵌入到引用中。二者配合，使宏作者可以描述「带空洞的代码模板」。

```
`{ if $cond { $body } }      // $cond 与 $body 是空洞
`(($x) * 2)                  // 表达式形态：$x 位置由实参填入
```

插值遵循与插值字符串 h"..." 一致的直觉：看到记号就知道此处会被替换，其余部分是字面结构。

## 12.4 编译期计算

常量表达式在编译期求值。这使得查表、派生量、配置校验等可以在产物生成之前完成，运行期只留下结果。

```
const TABLE_SIZE = 1 << 10
const MASK = TABLE_SIZE - 1

macro assert_const(cond: code) -> code {
    // 在编译期检查条件，不满足则报错
    return `{ }
}
```

## 12.5 使用边界

宏适合表达那些「写法重复但语义简单」的结构：断言、日志、资源收尾、派生实现。它们不适合承载主要业务逻辑，原因是宏展开后的代码在调试器里的位置信息不如直接书写的代码直观。

- 用宏消除重复的样板，不用宏制造新的语言
- 宏的展开结果应当可以用手写代码等价表达
- 宏的报错信息应当指向调用点，而不是展开内部

> 标准库的断言、日志与测试宏都遵循同一形态：调用点简洁，展开结果平凡，报错指向调用处。

## 12.6 两种宏

宏有两种形态，区别在于宏作者面对的是「语法结构」还是「书写形式」。

| 形态 | 作者面对的 |
| --- | --- |
| 函数式宏 | 参数与返回值都是代码片段类型。宏体是一段普通代码，在编译期执行，对片段做任意加工。 |
| 模板宏 | 参数是书写形式本身。宏体给出展开模板，按形参与实参的对应关系替换。 |

函数式宏适合需要逻辑判断的展开——按参数形态决定生成什么；模板宏适合形态固定的展开——把一段样板中可变的几处抽出来。选择依据是作者手上需要的是结构还是形式。

```
// 函数式：形参类型是代码片段，宏体在编译期执行
macro unless(cond: code, body: code) -> code {
    return `{ if !(@cond) { @body } }
}

// 模板：直接给出展开形态
macro twice(stmt) {
    @stmt
    @stmt
}
```

### 两种宏的写法对照

宏体里的代码片段需要与「原样保留的结构」区分开。准引用记号表示「这是一段代码结构，先不要执行」；插值记号表示「此处嵌入一个片段」。二者配合，宏作者可以精确描述「哪些部分固定、哪些部分由实参填入」。

```
// 准引用：包住一段代码结构
var tpl = `{ if @cond { @body } }

// 插值：把片段嵌入准引用
`(($x) * 2)      // $x 位置由实参填入
```

模板宏里以特定前缀命名的形参表示卫生临时名：展开时为它生成一个不会与外层名字冲突的新名字，因此模板内部引入的临时变量不会意外遮蔽使用处的变量。

## 12.7 声明注解

注解写在声明之前，为工具链提供附加信息。它不改语言的语义，但可被编译器、文档工具、测试框架等读取。

```
@priority(3)
@deprecated("改用 fast_path")
pub func slow_path(input: i64) -> i64 { ... }
```

注解可叠加，参数为字面量。把这类信息写成注解而不是注释，是因为注解可以被程序读取——检查工具能据此施加规则，文档工具能据此生成索引，而不只是给人看。

### 12.8 条件编译

同一份源码可能要在不同平台或不同构建配置下产出不同结果。条件编译按编译期已知的条件选择性地纳入一段声明。

```
#cfg(os = linux)
pub func path_sep() -> string { return "/" }
#else
pub func path_sep() -> string { return "\\" }
#end
```

条件的键是编译期已知的一组：os（操作系统）、target（目标平台）、debug / release（构建档）。条件在编译期求值，未选中的分支不参与检查也不进入产物——因此在某一平台上无法通过检查的代码，只要不属于该平台的分支，就不会妨碍编译。这一点与运行期分支有本质区别。

---

## 第 13 章 所有权与移动

每个值都有明确的归属。绝大多数时候归属是显然的——值属于声明它的那个变量。当值需要在变量之间转手时，语言要求这件事被写出来，而不是悄悄发生。

### 13.1 默认按值

赋值与传参默认按值：新位置得到一份独立的值，原位置继续有效。两个变量此后互不影响，读写其中一方不会改变另一方。

```
var a: i64 = 1
var b = a
b = 2
println(to_string(a))   // 1
```

按值形参的写入是不可观测的。被调方写入的是自己保有的那一份：函数返回后修改随之消失，调用方手里的值从未改变。写出来像是修改、运行时什么也没有发生——这类沉默不是语言愿意给的，因此对按值表形参的直接写入——下标写入、追加（见 3.11）以及以该形参为接收者的原位操作——在编译期检出为「不可观测写入」。

```
func fill(t: table<i64>) {
    table_push(t, 1)   // 错误：不可观测写入（按值形参）
}
```

要改动调用方的值，把形参声明为 ref——写回原对象正是 ref 的语义（见 13.4）；若希望调用方显式接住修改，也可以让函数返回合并后的新值，由调用方赋回。

```
func fill(ref t: table<i64>) {
    table_push(t, 1)   // 写回调用方的表
}

var xs = table_new_i64()
fill(xs)               // xs 变为 [1]
```

## 13.2 显式移动

当不需要保留原变量、且希望避免复制开销时，用移动记号把所有权转出。此后原变量处于「已移动」状态。

```
var big = collect()
var owned = big        // 复制或转移，按类型语义
consume(owned)
```

参数声明为 move 时，调用即转移：形参约定本身就表达了「这个值交给被调方」，调用方不必在调用点重写记号。

```
func take(move t: table<i64>) -> i64 { return len(t) }

var xs: table<i64> = []
xs <- 1
var n = take(xs)
// 此后 xs 不可再用
```

## 13.3 已移动状态

变量被移出之后再次使用会被检出。这条检查覆盖直接读取、再次传入、以及作为表达式的一部分使用。

```
var n = take(xs)
println(to_string(len(xs)))    // 错误：变量已移出
```

检出发生在编译期，因此这类错误不会在运行期以「读到已释放对象」的形式出现。需要保留原值继续使用时不使用移动，改用按值或借用。

## 13.4 借用与转移的取舍

三种做法对应三种需求，选择依据是「调用方还需要这个值吗」。

| 需求 | 做法 |
| --- | --- |
| 调用方不需要继续用 | 声明 move，把所有权交给被调方 |
| 调用方继续用，只读取 | 声明 immut，避免复制 |
| 调用方继续用，且要被调方修改 | 声明 ref，写回原对象 |
| 双方各持一份，互不影响 | 按值，接受一次复制 |

## 13.5 复制与克隆

按值传递对小的标量代价为零；对表这类动态结构则是一次真实的数据复制。需要明确表达「我要一个独立的副本」时使用克隆，使复制的成本在代码里可见。

```
var snapshot = xs.clone()
```

> 可写借用不产生副本，因此能修改原对象；但它有生命周期限制——借用不能被闭包捕获带走（见第 5 章），也无法越过函数边界长期存在。这组限制共同保证借用指向的对象始终有效。

# 第 14 章　界面元素块与图语法

有两类结构，用函数调用写出来会显著失去可读性：嵌套很深的界面元素树，以及需要表达拓扑关系的图。语言为二者提供专门的记号，使代码的形态接近它描述的结构本身。

## 14.1　元素块的写法

元素块以 @ 记号引入，块内书写元素与属性。元素之间的嵌套关系由书写的层次表达，不需要括号配对。

```
@ window "设置" {
    size: 480 x 320
    @ panel {
        @ label "服务地址"
        @ input host: "localhost"
        @ row {
            @ button "保存" on_click: save
            @ button "取消" on_click: cancel
        }
    }
}
```

这段写法与它描述的界面同构：外层窗口包含一个面板，面板包含一行文本、一个输入框与一个按钮行。改结构时改动的位置与视觉上的位置一致。

## 14.2　属性与绑定

元素的属性写在元素之后，形式为名称与值。值可以是字面量、变量或表达式；写表达式时用花括号界定，形式与插值字符串 h"..." 中的花括号一致。

```
@ label title
@ progress value: done * 100 / total
@ input host: config.host
```

属性可以分行书写，也可以放在同一行。分行时每个属性独占一行，便于对齐与增删。

## 14.3　事件

事件用 on_事件名 的形式绑定到一个函数值。绑定接受具名函数，也接受短闭包。

```
@ button "重试" on_click: retry
@ button "清空" on_click: t -> clear_all()
@ input host: "localhost" on_change: host_changed
```

事件处理函数是普通函数，参数与返回类型由事件类型决定。由于函数值遵循第 5 章的捕获规则，事件处理只携带值快照，不持有对界面外部状态的可写引用。

## 14.4　数据与界面

界面元素可以引用所在作用域的值。值的更新触发界面刷新，刷新范围由引用关系决定——读取了某个值的元素才会重建，其余部分保持不动。

```
func view(state: AppState) -> i64 {
    @ window "计数器" {
        @ label "计数: " + to_string(state.n)
        @ button "加一" on_click: bump
    }
    return 0
}
```

## 14.5 图语法

图语法用于描述节点与它们之间的连接。节点声明与边声明分开书写：先给节点起名，再用箭头描述连接方向。

```
graph Pipeline {
    node fetch
    node parse
    node store

    fetch -> parse
    parse -> store
    store -> fetch
}
```

箭头方向即数据或控制流的流向。需要给边附加条件或权重时，在箭头之后书写。

```
    parse -> store when valid
    parse -> fetch   when invalid
```

图作为值可以构造、组合与执行：构造产生图对象，组合把两个图按同名节点连接起来，执行按拓扑顺序推进。由于图是不可变的，组合不会修改原图，而是产生新图——这使得图可以在多个调用点之间安全共享。

> 元素块与图语法都不是独立语言，而是同一套表达式的书写形态。二者内部可以出现普通表达式、函数调用与闭包，也可以被普通代码构造出来。

## 14.6 界面单元

界面由若干「单元」组成：页面级的视图、可复用的组件、以及样式与主题的声明单元。单元是文件的顶层声明，与函数、结构并列。

| 单元 | 作用 |
| --- | --- |
| 视图 | 页面级界面单元。有输入（属性）与输出（动作契约），把外部状态映射为界面结构。 |
| 组件 | 可复用界面单元。与视图同构，额外支持内容插槽，使调用方可以填入子内容。 |
| 样式 | 选择器规则与分层声明。见 14.8。 |
| 主题 | 设计变量的取值表。见 14.8。 |
| 动作 | 交互契约：只声明有哪些交互，不写实现。实现由外部提供。 |

把界面写成声明而不是构造调用，目的是让界面结构与它的视觉形态保持同构：嵌套关系由缩进表达，

层次一目了然。

```
view Counter(count: i64, history: table<string>) -> (Dec, Inc, Reset) {
    @Column {
        @Text "计数：" + to_string(count)

        if count > 2 {
            @Badge "偏大"
        }

        for h in history {
            @Text h .muted
        }

        @Row {
            @Button "-1" .lg -> Dec
            @Button "+1" .lg -> Inc
        }
    }
}
```

视图的入端口是属性，出端口是动作契约。出端口决定这个视图可以向外部发出哪些交互——使用方据此提供实现，视图本身不关心交互如何被处理。

## 14.7 元素与控制流

元素用 @ 引入，写在界面单元的块内。元素语句之间可以穿插「原生控制流」：条件、遍历与分支直接写在界面里，表达「按条件显示」「按集合重复」——不需要为界面另造一套条件指令。

元素的内容形式有两种：字符串字面量可以省略括号，其他表达式保留括号。

```
@Text "直接写字面量"          // 省略括号
@Text(to_string(count))      // 表达式保留括号
@Text "计数：" + to_string(count)
```

控制流在界面里与在函数里语义完全一致：条件为真才纳入该元素，遍历按集合逐项展开。这种一致性免去了「界面里的条件与代码里的条件规则不同」这类需要额外记忆的地方。

## 14.8 修饰与事件

元素可以带修饰链，用于给出外观与档位。修饰分两类：外观钩子与语义档位。

- 外观钩子：含义由样式表给出。结构文件里不出现颜色、字号这类具体取值，只出现名字。
- 语义档位：含义由组件库给出，表示尺寸、密度等结构性选择。

```
@Text "标题" .title
@Text "说明" .muted
@Button "+1" .lg -> Inc
```

事件用箭头把元素连到动作：箭头右侧是动作名，表示「这个元素被触发时发出该动作」。控制流的流向与数据流的写法一致，因此界面的交互路径与数据变换路径用同一套阅读方式。

### 分层与作用域

样式按层组织，层的先后由声明顺序决定：后声明的层覆盖先声明的层；同层内按书写顺序，后者胜出。语言不做选择器权重计算——同一规则在不同层级的胜负关系一眼可见，不需要推算特异性分数。

```
@layer base, components, overrides

@scope Card {
    .title { 前景: 表面色 }
}
```

主题是一张变量取值表，样式规则引用变量而不写死取值。因此换肤只需换主题，样式规则不动。

```
@theme dark {
    表面色 = 深灰900
    强调色 = 蓝400
}
```

这样的分工使三件事各自独立：结构决定有什么，样式决定长什么样，主题决定用什么色。改动其中一项不影响另外两项。

# 第 15 章 工具链与语言服务

语言规范定义了什么是对的；工具链让书写、检查与维护这件事变得可行。本章说明编译、诊断、格式化与文档生成的行为约定。

## 15.1 编译流程

编译分为若干阶段，每阶段有明确的输入输出，可以对中间产物单独检查。

| 阶段 | 职责 |
| --- | --- |
| 预处理 | 处理头部声明、导入关系与编译期开关 |
| 词法与语法 | 切分记号、构建语法结构，报告语法错误 |
| 语义分析 | 类型检查、约定检查、所有权与移动检查、捕获面检查 |
| 中间表示 | 下沉为与机器无关的中间代码 |
| 优化 | 常量折叠、内联、单态化展开、无用代码消除 |
| 生成 | 产出目标代码并与运行时链接 |

语义分析阶段的检查顺序是确定的：类型在前，约定居中，所有权与捕获在后。固定的顺序使得同一次编译里的报错不会因为检查次序而变化。

## 15.2 命令行

编译器的常用调用形态是一致而可组合的：源文件、输出、开关。

```
tiec app.tie                    // 编译为可执行文件
tiec lib.tie -o lib.a           // 编译为静态库
tiec app.tie --emit-ir          // 只产出中间表示
tiec app.tie --check            // 只做检查，不产物
tiec app.tie --emit-pdf         // 随产物输出结构报告
```

优化级别与目标平台属于命令行开关，不写在源文件里。这样同一份源码可以在不同目标下编译，而不必为每个目标维护一份文件。

## 15.3 诊断信息

每条诊断有稳定的编号、明确的类别与指向具体位置的坐标。编号使得错误可以被引用、被加入回归语料，而不必复述整段消息。

```
error[E00387] main.tie:12:5: 命名实参调用 'connect' 缺少第 2 个形参 'port' 的值
warning[W00008] main.tie:3:5: 变量 'tmp' 未被使用
```

诊断的撰写遵循三条约定：指出问题所在的位置；说明期望与实际的差别；在能给出修法时给出修法。

## 15.4 格式化与静态检查

格式化工具按统一规则重排代码，使风格差异不进入评审。它的输出是确定的：同一份代码格式化两次结果相同。

静态检查工具在编译之外给出建议，覆盖可能的疏漏——未使用的变量、被遮蔽的名称、恒真的条件、可以简化成管道表达式的嵌套调用。

```
tiefmt src/          // 就地格式化
tielint src/         // 静态检查
```

## 15.5 文档生成

以三条斜线书写的文档注释被工具提取，与声明一起生成 API 文档。文档注释紧接在声明之前，就属于那个声明；写在函数体内部则退化为普通注释。

文档注释的第一行是摘要，其余部分与正文使用同一套排版记号，只是写成文本形态。

| 排版构件 | 文档注释里的写法 |
|---|---|
| 摘要 | 首行 |
| 段落 | 空行分隔：一行只有三条斜线 |
| 代码块 | 整行缩进四空格 |
| 清单 | 每项以 - 开头 |
| 提示 | 以 > 开头 |
| 小标题 | 以 # 开头，个数表示层级 |

```
/// 求两数的最大公约数。
///
/// 示例：
///     gcd(12, 18)   // 6
///
/// > 入参为零时返回另一数。
pub func gcd(a: i64, b: i64) -> i64 { }
```

文档注释写的是签名表达不了的东西：这一项为什么存在、取值范围与相互约束、边界行为、失败与副作用方式、与谁相关。

与之相对，参数的说明不重复进文档注释：类型、可空、默认值、ref / immut / move 都是签名的一部分（见 16.13）。重复书写会留下第二处真相——签名改了而文档未改，文档就开始误导读者，这比没有文档更糟。

## 15.6 交互式环境

交互式环境接受表达式并立即求值，用于试验标准库函数与语言构造。它与编译使用同一套语义实现，因此交互环境里的结果与编译产物的行为一致。

```
$ tie
> var xs = [3, 1, 2]
> xs -> sort
[1, 2, 3]
```

## 15.7 语言服务

语言服务为编辑器提供补全、跳转、悬停与实时诊断。它按文件增量重算，不要求整工程重新编译；诊断的编号与命令行输出一致，因此编辑器中的提示与构建日志可以直接对照。

> 工具链与语言本身共用同一份实现：解析、类型检查与诊断由同一套代码提供，这保证编辑器里看到的结论与编译结果不会出现分歧。

---

# 第 16 章 速查表

本章把全书的形态汇总成查阅表，供书写时快速定位。语义细节见对应章节。

## 16.1 文件与角色

```
type tie                // 泛型入口（Type 角色）
type tie<logic>         // 逻辑代码（默认）
type tie<class>         // 类库
type tie<tsc>           // 脚本
type tie<data>          // 纯数据
type tie<ui>            // 界面
type tie<db>            // 数据库
type tie<port>          // 接口
type tie<ir>            // 中间表示
type tie<test>          // 测试
type tie<bench>         // 基准
type tie<zd>            // 结构化二进制数据
type tie<tsh>           // 终端脚本
```

## 16.2 声明形态

```
var x = 1                       // 变量
const K = 10                    // 常量
func f(a: i64) -> i64 { }       // 函数
pub func g() { }                // 公开函数
struct S { var n: i64 = 0; }    // 结构
enum E { A(x: i64) }            // 枚举
namespace ns { }                // 命名空间
port P { pub func m(self) }     // 接口
impl P for S { }                // 实现
actor A { var n: i64 = 0; }     // 状态单元
macro m(x: code) -> code { }    // 宏
import "./m.tie" as m           // 导入
extern func e(a: i64) -> i64    // 外部函数
```

## 16.3 参数约定

```
func f(a: i64)                  // 按值
func f(ref t: table<i64>)       // 可写借用
func f(immut t: table<i64>)     // 只读借用
func f(move t: table<i64>)      // 所有权转移
func f(a: i64, b: i64 = 10)     // 默认值（须连续居尾）
func f(a: i64, b: i64, c: i64)  // 调用：f(a: 1, c: 3) 命名并跳过 b
func f(rest: ...i64)            // 变长参数
```

## 16.4 表达式记号

```
a -> f                  // 数据流管道
t -> t + 1              // 短闭包（要求函数值的位置）
x in m                  // 成员判定
x not in m              // 成员取反
v ?: 10                 // 安全默认值
v?                      // 存在判定
\p                      // 解引用
n %%= 1                 // 原地取余并写回
a ** b                  // 幂
a **? b                 // 带检查的幂
0..10   0..=10          // 范围
```

## 16.5 语句形态

```
if cond { } else { }
while cond { }
for i in 0..10 { }
for x in xs { }
for c in s.chars() { }
lbl: for i in 0..10 { break lbl }
switch v { case 1: }
return v
yield v
break    continue
unsafe goto #tag
panic("msg")
```

## 16.6 常用内建

```
var t: table<i64> = []        // 空表：字面量即构造
var m: map<i64> = [:]         // 空映射
t <- v                        // 追加（返回自身）
println(s)              // 输出一行
len(t)                  // 元素数或字节数
to_string(v)            // 转文本（自由函数：接收者不限类型）
len(s)  str_char(s, i)  // 码点数；取第 i 个码点
str.trim(s)  str.split(s, sep)  str.join(xs, sep)  str.contains(s, sub)
seq_sort(t)  seq_reverse(t)  seq_distinct(t)   // 集合操作（见 2.20）
seq_group_by(key, t)  tbl_cat(a, b)          // 分组与拼接
iter_map(f, xs)  iter_filter(p, xs)  // 迭代器适配（见 6.11）
iter_take(n, xs)  iter_zip(a, b)  iter_chain(a, b)
r.wrap_err(msg)  e.cause()  e.root_cause()   // 错误链（见 8.7）
Ok(v)  Err(e)  Some(v)  None          // 构造名默认可见（见 9.4）
file_read(p)  file_write(p, s)
byte_read(p)  byte_write(p, t)   // 二进制安全
load_library(n)  get_proc(h, n)  dyn_call(p, ...)
```

## 16.7 诊断编号

诊断编号由类别字母与五位数字构成：E 打头的是错误，W 打头的是警告。编号一经分配即不再变动，新增诊断取新号、不复用旧号。

数字段不编码类别——编号按登记顺序连续分配，分类信息由编号表另行登记，因此按类别检索应当查表，而不是按数字区间推断。

| 区间 | 类别 |
| --- | --- |
| E | 错误：代码无法编译，必须修正 |
| W | 警告：可疑但合法，按默认设置提示 |

> 编号稳定是工具链的前提：编辑器、CI 与回归语料都按编号引用诊断，重排编号会同时破坏这三者。曾经规划按类别划分数字区间（语法与词法、内建调用、类型与调用、导入与作用域、借用与移动、并发与宏各占一段），该规划未采用：区间需要预留空号，而预留与「连续分配、永不复用」在长期演进中彼此冲突。类别检索改由编号表的类别字段承担。

## 16.8 设计要点回顾

本书的规则可以收敛成若干条彼此呼应的立场，它们共同构成语言的性格。

- 参数约定写在声明处，四种划分互斥且完备——读签名即知实参的去向
- 命名实参可跳过带默认值的参数，使「只改一处」的调用最短
- 类型系统是静态的，标注是可选的：pub 边界必须标注，边界之内交给编译时推导
- 链式分工一条划清：点调用是对象自身的行为，箭头把值送进外部变换
- 闭包捕获对容器按引用、对其余按值快照；共享可变状态只可能经由容器捕获出现
- 并发通过消息与任务图表达，共享不通过共享内存偶然发生
- 不安全操作写在标记区内，审计可按文本搜索完成
- 常见的书写意图有专有记号：管道、短闭包、成员判定、追加、原地修改、安全默认值

## 16.9 字符串与数字

```
"普通字符串，花括号即字面量"
h"{插值}" 花括号内求值
h"{x:08.2f}" 插值带格式说明符
r"原始字符串无转义"
"""多行字符串，按闭合引号缩进剥离"""
'A'                          // 字符

42  0xFF  0b1011  0o755  0t1T0      // 五种进制
1_000_000                          // 下划线分组
3.14  1.5e3                        // 浮点
42i32  0x80u16  1.5f32  0.1d       // 类型后缀
-1t  0t  1t                        // 三值
```

## 16.10 运算符速览

```
a + b   a - b   a * b   a / b   a % b     // 算术；容器上 + 为拼接、* 为重复
a \ b   a %% b                           // 整除与成对取余
a ** b   a **? b                         // 幂；带检查的幂
a +? b   a -? b   a *? b   a <<? b        // 带检查的算术与移位
a == b   a != b   a < b   a <= b         // 比较（可链式）
a in xs   a not in xs                    // 成员判定
v is T                                   // 类型判定（动态值，见 3.16）
cond && cond   cond || cond   !cond       // 逻辑
a << b   a >> b   a & b   a | b   a ^ b   ~a   // 位运算（整数）
a | b   a & b   a - b   a ^ b           // 容器：并、交、差、对称差
x = v   x += v   x \= v   x %%= v        // 赋值与复合赋值

[x * x for x in xs if c]   // 集合综合（见 3.17）

v -> f           // 管道
v -> [i]         // 管道进下标
v -> .field       // 管道进字段
v -> { x -> e }   // 块管道（x 为绑定名）
x <- v           // 反向赋值
t <- v           // 容器追加（返回自身，可链式）
(a, b) <- t      // 解构
v with { k: x }   // 不可变更新
opt ?   opt ?: d   obj?.f   obj?[i]       // 可空链
xs[1..3]   xs[..2]   xs[2..]   xs[..]     // 切片
await f                                   // 延后取得异步结果（见 10.6）
\p                                       // 解引用（不安全）
```

## 16.11 语句速览

```
if c { } else if c2 { } else { }
var x = if c { a } else { b }             // 条件取值（见 4.13）
if let Some(v) = opt { }
while c { }
for i in 0..n { }
for x in xs { }
for c in s.chars() { }
lbl: for i in 0..n { break lbl }
switch v { case 1: ... default: ... }
var x = switch v { 0 -> a, _ -> b }       // 分支即表达式
var (a, b) = pair
var (a, ...rest) = xs
guard c else { return }
defer { cleanup() }
try { x = f()? }
return v    yield v    break    continue
panic("msg")
unsafe goto #tag                          // 配 #[tag.tag] 标签
unsafe { }                                // 不安全块
```

## 16.12 声明速览

```
[pub] [unsafe] [async] [inline] func f(a: i64) -> i64 { }
func f(x: i64) -> i64 = x * 2                // 等号体
struct S<T> extends B { var f: T = d; }
enum E { A  B(x: i64) }
namespace ns { }
port P { pub func m(self) -> i64 }           // 名义接口
interface I { pub func m(self) -> i64 }      // 结构接口
impl P for S { pub func m(self) -> i64 { } }
actor A { var n: i64 = 0;  pub func m() { } }
macro m(x: code) -> code { }
alias M = table<f64>
extern func e(a: i64) -> i64
import "m.tie" as m
import "m.tie".{ a, b }                      // 选择性导入
pub import "m.tie"                           // 重导出
using ns.sub
var g: table<i64> = []
const K: i64 = 10
@注解(3)  #[属性]  #cfg(cond)               // 注解与条件编译
view V(p: i64) -> (Act) { @Column { } }      // 界面单元
```

## 16.13 形式与分工

把全书反复出现的判断收束成一张对照：遇到「这个信息该写在哪」，查这张表。

| 要表达的意思 | 写在哪 |
| --- | --- |
| 这个参数能否被改 | 参数名前缀：ref / immut / move |
| 这个名住在哪里 | 住在实体里的用点；外面的用箭头（见 3.11） |
| 改写形参的改动如何交回 | ref 写回原对象，或经返回值合并（见 13.1） |
| 这个值要不要保留 | 不要保留用 move，只读用 immut |
| 这一项的默认取值 | 参数默认值（须连续居尾） |
| 调用时这一项是什么 | 命名实参 |
| 这个类型怎么用 | 类型标注与别名 |
| 返回值是什么 | 函数签名（必写） |
| 这一项为什么存在 | 文档注释（见 15.5） |
| 这个文件是什么 | 文件头 type tie<角色> |
| 哪里允许不安全操作 | 文件头或函数上的能力域标注 |
| 这块代码属于哪一层 | 命名空间与可见性 |
| 这条消息该给谁 | 执行体 actor |
| 这一步依赖什么 | 数据流图或管道 |
| 这个结构长什么样 | 界面单元的 @ 元素块 |
| 按条件在两个值中取一个 | if 表达式（见 4.13） |
| 动态值的实际类型 | 类型判定 is（见 3.16） |
| 延后取得异步结果 | future 凭据与 await（见 10.6） |
| 这个取值能不能缺 | 可空链 ? / ?: / ?. / ?[ |
| 溢出怎么办 | 带检查的运算 +? 或 checked_* |
| 越界怎么办 | 读报错、写扩容；切片按边界截断（见 3.15） |
| 这段代码在别的平台 | 条件编译 #cfg |

这张表也可以反向使用：阅读他人代码时看到某个位置写了东西，就知道作者在表达哪一类意图。

# 第 17 章　生态协议与格式

语言之外，tie 生态由四个公开契约联结：td 明文数据格式（tie:data）、zd 二进制序列化（tie:zd）、tink 帧协议（数据流互联）与 tsha1 哈希家族（内容指纹）。它们共同构成一条语言无关的通路——任何语言的组件只要实现这些契约即可接入生态。本章逐一给出规范基准；实现以此为准，设计文档与代码注释不得与之冲突。

## 17.1　td 与 tie:data

td 是生态的明文数据格式，文件角色为 tie<data>。它的核心约定只有一条：正文就是 tie 的表字面量——不引入 JSON 风格的独立语法，会写 tie 就会写数据。

- 正文为裸表 [ … ]，或带可选表名的形态 name = [ … ]（无 var 关键字，表名可省略）。

- 角色头 type tie<data> 可选：省略头部、正文直接写表同样成立（与带头部等价，加载器一视同仁）。
- 值支持字符串（UTF-8，含中文）、整数、浮点、布尔、数组、键值表与任意嵌套。
- 注释用 //，与 tie 一致。
- 解析由编译器背书（tiec 语法层直接支持），标准库不承担解析职责。

```
// pack.data.tie —— type tie<data>
[
    "season": "S13",
    "season_id": "2026-08",
    "author": "tie-lang",
]
```

td 有两种载体形态：可读的明文（tie:data，面向开发者与版本对比）与等价的二进制（tie:zd，由编译器压缩产出，面向机器分发）。二者表达同一份数据，语义不变，加载器对二者的审计方式一致。编译器子命令 --compress-data 完成 td 到 zd 的单向编译：

```
tiec --compress-data <in.data.tie> -o <out.zd>
```

编译采用深度优先平铺：表字面量树按 DFS 序展开为元素序列，以六个平行列描述——kind（0 表/数组、1 字符串、2 整数、3 浮点）、key（键名，无键为空串）、value_i64、value_f64、value_str（标量值按 kind 取对应列）、child_count（子元素数，消费者据此推进子树遍历）。六列编码为 zd record 的六个字段，字段号 1 至 6 固定且只追加；布尔归整数列（true 为 1）。解析失败即报错退出，不产出 zd。

## 17.2 zd：二进制序列化

zd 是语言无关的二进制序列化格式——tie 生态的统一数据载体。任何语言可独立实现；本章即规范基准。服务场景：数据包容器、快照、td 编译产物、tink 统一 IO 帧、生态数据交换。扩展名一律 .zd。

### 头部与版本

v2 头部共 10 字节：前 7 字节为魔数 TIEDBZD，其后两位为 base-48 版本数字（各 1 字节，存数值 0..47；字符集 std base48：0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKL，L 即 47），最后 1 字节为 flags。版本值 version = d1 × 48 + d2，v2 写作字节 [0][2]，v3 写作 [0][3]。

识别规则：9 字节且魔数匹配、两位版本数字均不越界，按 version 判定；8 字节（魔数 + 0x01）是 v1；其余非法。版本沿 d1 × 48 + d2 单调递增。

兼容链自 v3 起调整：v3 实现的读取义务为 v2 与 v3——v2 的每个合法文件都是 v3 的合法文件，字节布局不变；v1 从读取义务降级为可选库能力，写方自 v3 起禁写 v1，存量 v1 文件经一次性转换迁移（宽限期一个次版本周期）。v2 的新类型对 v1 消费者不可见的规则不变：record 字段号只追加，未知字段与未知 ext 类型整体跳过。

flags 字节固定存在（缺省 0），v2 五位：bit0 字符串字典、bit1 列式容器、bit2 ext 扩展、bit3 流式分块、bit4 压缩变体声明。压缩算法不内嵌——头部仅声明变体（zstd/lz4），解压由读端承担。v3 新增两位：bit5 索引（文件尾存在索引 footer，为文件的规范组成部分）、bit6 图容器（文件含图容器段，纯提示位）。

类型系统

| 类型 | 编码 |
| --- | --- |
| i64 / u64 | fixint 与定宽大端；正 0x00..0x7f、负 0xe0..0xff、定宽 0xd0..0xd3（u64 为 0xcf/0xd3）。 |
| f64 | 0xcb + 8 字节大端 IEEE 754 位模式直写（bitcast，不做数学分解）。 |
| bool | 0xc2 / 0xc3。 |
| string | fixstr/str8/16/32 + UTF-8；v2 增加字典引用形式（池内索引 varint，需 flags 字典位）。 |
| bytes | 0xc6 族 + 长度 + 原始字节（图标/像素/任意载荷）。 |
| array / map | fixarray/16/32 与 fixmap/16/32；自描述，元素带类型标签。 |
| null | 0xc0；区分「缺失」与「空值」。 |
| ext | 0xd7 + 类型标记 + 长度 + 载荷；tie 特有类型（char、trit）与 code（tie-IR）一律走 ext 子类型，不进核心类型系统。 |

每语言实现必须支持核心类型；ext 可选择性支持，未知 ext 类型整体跳过。

实现纪律

以下是实现的规范性要求，不是可选优化：编码一律写入单增缓冲（append-only），消除连续拼接的二次方开销；f64 用位模式直写；字符串解码一次批量构建 UTF-8；支持预分配与批量原语；解码字符串与 bytes 可返回区间引用（offset + length）实现零拷贝读取；大容器可带偏移表（每 N 元素记录字节偏移）以支持快速跳读与随机定位。

格式能力

- 列式容器：列数 + 每列类型 + 每列长度 + 各列值，供大批同构 record（表格型数据）压缩体积、提升局部性。
- 字符串字典：容器可带字符串池段，重复字符串只存一次（池内引用为 varint 索引）。
- schema 段：容器携带字段名与类型的自描述信息，任意语言可反序列化到对象。
- code 承载：tie-IR 经 ext 子类型内嵌，载荷对齐 TIEIR 七段格式（魔数/模块头/字符串池/符号表/IR 主体/导出表/span）。
- 流式帧：帧带长度边界与 CRC 校验（见 17.3），损坏帧拒绝，不静默传递。
- 内容哈希段：容器内嵌内容哈希（tsha1 族，见 17.4），读端做完整性校验。

> zd 只管序列化：压缩算法、加密签名、数据库与索引都不在格式之内，它们是上层策略与上层组件的职责。

v3：索引 footer 与列式编码族

v3 把随机访问标准化为文件尾索引 footer：魔数 ZD3FT + 索引段偏移 u64 大端 + 索引段长度 u64 大端 + 覆盖索引段的 CRC32（IEEE），共 25 字节。索引段是段表——段数与各段的类型、偏移、长度、可选名称——footer 存在时（flags bit5），消费方必须经索引定位段，不得假设段序。段类型初始集合：0 数据体、1 字符串池、2 列式容器、3 schema、4 图容器、5 压缩载荷、6 自定义。

列式容器为每列引入编码声明，写在列头。初始编码族四件：0 plain（原值序列）、1 RLE（游程 + 值，游程不小于 1）、2 delta（首值 + varint 差分，仅整型列）、3 字典（列内字典 + varint 索引序列）。编码是列级声明、纯表示层：解码后的值域与 plain 完全一致，读端按声明解码。列式、编码族与字符串池三者正交，可同文件并用。

### v3：schema 演进

schema 段新增 schema_id——内容的 tsha1f 指纹，同一 schema 的所有文件 id 相同，读端据此协商与缓存。字段生命周期三态：active 正常；deprecated 读取方必须仍能读、写方禁止新写；removed 物理删除，且仅允许在字段进入 deprecated 一个版本周期之后。字段号永不复用：removed 后其编号作废，新字段取新号。跨版本读取按字段号对齐，未知名与未知类型字段跳过；读端不支持文件的 schema_id 时按可跳过规则降级读取，不静默改写。

### v3：标准类型

三个高频类型自 v3 起以固定 ext 类型标记升格为标准类型：v3 实现强制支持，v2 读取方按未知 ext 整体跳过。timestamp（标记 1）：8 字节 Unix 纪元秒 i64 大端 + 4 字节纳秒 u32 大端。decimal（标记 2）：符号 1 字节 + 十进制数字串 UTF-8 + 指数 i8，值为 ±数字 × 10^指数。uuid（标记 3）：16 字节 RFC 4122 位模式。三者的取舍理由一致：以固定标记取得「实现义务 + 互操作保证」的核心地位，同时不与 v2 既有标记碰撞。

### v3：图容器

图容器是 v3 新段类型（段类型 4），同时可作为 ext 子类型（类型标记 0x47）嵌入任意 zd 值位置。结构：声明字节（bit0 有向、bit1 节点 id 形态）+ 节点数 + 节点表 + 边数 + 边表。节点 = id + 任意 zd 载荷 + 可空属性 map；边 = from 与 to 的节点引用、可覆盖容器方向的边头字节、可选标签 string、可选权重 f64 大端、可选属性 map。

约束：节点 id 唯一；边引用必须指向存在的节点；自环与多重边合法；空图合法。大图的节点表与边表应当用列式容器加编码族承载——id 列用 delta、标签列用字典。tie 侧图的边与节点操作（见 10.9）产生与消费此容器；节点载荷为任意 zd 值，故可携带 code 等 ext 载荷。

## 17.3 tink 帧协议

tink 是生态的数据流互联契约：组件以独立进程存在（不被承载、不被嵌入），数据以 zd 字节流在组件间流动。接入门槛只有一条——实现统一帧协议。语言无关意味着 tie、Rust、C、C++、Python 的组件，乃至 AI 代理进程，都是平等节点。

统一 IO 契约是：调用形态为「模块.函数(字节进) → 字节出」；组件的标准输入输出上以「长度前缀 + zd 消息」为帧，帧带边界与 CRC 校验。任何语言只要在 stdin/stdout 上实现这一帧协议，即成为节点，不依赖编排器的内部实现。

帧协议三要素：长度前缀界定帧边界；载荷为完整 zd 消息；CRC 校验完整性。CRC 失败的帧必须拒绝并报错，不得静默传递损坏数据；管道某一段失败时须定位到段并以非零码退出。

| 形态 | 说明 |
| --- | --- |
| 管道编排器（主形态） | 按管道定义启动组件进程，把 A 的输出帧路由到 B 的输入帧。纯流式、轻量，替代脚本与临时文件。 |
| 常驻 hub（扩展） | 编排器常驻，组件连上注册端点，按路由表转发 zd 消息；引入 IPC 复杂度，按需启用。 |
| 库式互联层（扩展） | 编排器作为库链接进组件，组件间直接 zd 流互连，无常驻服务。 |

```
tink pipe "tiec:td2zd | jcc-pack:build" > out.jcc
```

三形态共享同一帧协议核心。编排器只做连接与路由：不执行任务调度、不管理并行、不替代编译链路本身——组件保持独立可执行，这是边界而非缺陷。

## 17.4 tsha1 哈希家族

tsha1 是 tie 的原生哈希家族，纯 tie 实现、零 unsafe，输出为 base-48 编码。家族含四个模型：f（R=12 双轨）、b（R=14 三轨）、x（R=16 四轨）、r（R=8 双轨半宽）；位长参数 n 取 2..144，名义输出位 k = n × log2(48) ≈ 5.585n。生态内职责：逐文件指纹用 tsha1f，包树根指纹用 tsha1x，zd 内容哈希段同族。

安全剖面与 SHA-3 同输出位档对齐，且实得安全位 = min(名义输出位, 状态位) = 名义输出位——状态随 n 构造保证状态位不低于输出位，因此不存在 SHAKE 式「输出长但安全被容量封顶」的缺口。代表档位：

| 档位 | 安全剖面 |
| --- | --- |
| n = 48 | 输出 268 位；碰撞界 2^134、前像界 2^268，对齐 SHA3-256 并高 6 位。 |
| n = 69 | 输出 385 位；碰撞界 2^192.7，对齐 SHA3-384 并高 0.7 位。 |
| n = 92 | 输出 514 位；碰撞界 2^256.9，对齐 SHA3-512 并高 0.9 位。 |

次安全性质为构造保证：抗长度扩展（计数器 MD 填充 + 64 位计数器逐块注入，无 0x80/长度后置）；轮常量由 SEED 经 SHA-256 计数器流生成，常量与实现可复现；输出截断语义与 base48 编解码一致。

> 边界声明：tsha1 未经过第三方密码学审计；「更优」仅指上表可量化剖面，不宣称设计强度超越 Keccak。用于正式安全用途前应经专业评审。性能上，位平面 ring 数学的每块成本约为 sha256 的 150 倍（算法成本模型差异，非缺陷），热点已按族内基准优化。

## 17.5 契约的协同

四个契约串联成一条完整链路：td 明文经编译器编译为 zd（17.1 → 17.2）；组件进程以 tink 帧协议互连，帧内是 zd 消息（17.3）；分发的完整性与来源由 tsha1 指纹与签名校验守护（17.4）。每一环都语言无关——生态由此向所有语言与 AI 代理开放。

> 契约的演进纪律与语言一致：魔数与版本位保证格式可判定地演进；record 字段号与 ext 类型只追加、不回收；帧协议以 CRC 与长度前缀保证可靠路由。任何改动都必须保持旧读取方可用。